

Bio-Medical Study

2021/03/04 ~

박지원

목차

- 1주차 : 디지털 영상의 기초
- 2주차 : Convolution 과 Filtering
- 3주차 : Skip
- 4주차 : 경계 검출
- 5주차 : Thinning(=세선화)
- 6주차_1 : 특징 검출을 이해하기 위한 행렬 지식
- 6주차_2 : 특징 검출 기법
- 7주차 : 다양한 특징 검출 기법

<1주차 : 디지털 영상의 기초>

1. 디지털 영상

1) 디지털 영상의 정의

= 모든 신호가 0과 1

- 아날로그 영상인 자연 영상을 컴퓨터에서 처리하기 위해서 디지털 영상으로 변환합니다.
- 디지털 영상은 2차원 행렬로 나타낼 수 있습니다.

ex) X-Ray , CT , MRI

2) 샘플링

= 2차원 공간을 일정하게 특정한 좌표 x,y 에서 존재하는 값으로 나타내는 과정

ex) 화소(=Pixel)

3) 해상도

= 2차원 공간에서 Pixel들의 x,y 축 개수의 비율

4) 양자화

= 각 Pixel의 밝기 값을 몇 단계의 밝기로 근사화 시키는 과정

- 양자화의 단계를 낮추면 영상의 화질이 떨어집니다.

2. Pixel 연산

1) 영상의 산술 연산

- 밝기의 수치는 0 ~ 255만 나타낼 수 있으므로 Overflow에 주의합니다.
- 255에 가까울수록 밝고 , 0에 가까울수록 어두워집니다.

① 덧셈

$$h(x, y) = f(x, y) + g(x, y)$$

- h(x,y)의 값은 255보다 작아야합니다.
- 다른 영상의 밝기가 더해지므로 기존보다 전체적으로 밝아집니다. 이를 보완하기 위해서 오른쪽의 식에 1/2를 하여 평균 연산을 수행하여 밝기 정도를 유지시킵니다.

② 뺄셈

$$h(x, y) = f(x, y) - g(x, y)$$

- 뺄셈의 반대 개념입니다.
- 두 영상의 밝기의 차이를 나타내기 위해 절댓값으로 표현한 차이 연산을 이용합니다.

③ 곱셈

- 영상의 밝기 값에 상수를 곱해서 영상의 밝기를 변경합니다.

2) 영상의 논리 연산

- AND , OR 연산 : 이산 수학에서 배운 논리 연산을 Pixel 연산에 적용시킵니다.

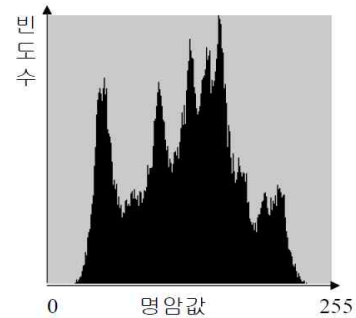
3. 히스토그램

1) 히스토그램의 정의

= Pixel이 가진 명암 값의 빈도수에 대한 막대그래프



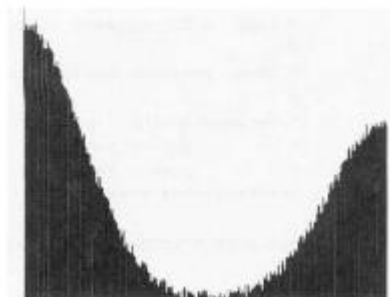
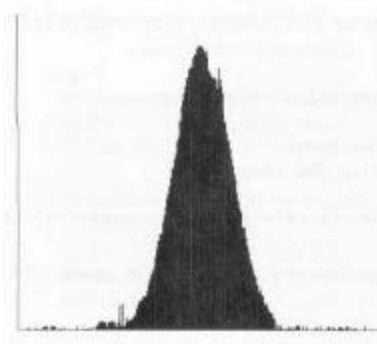
원 영상



히스토그램

2) 영상의 화질을 개선하는 방식 : 히스토그램 평활화 (=Histogram Smoothing)

<명암대비 스트레칭>



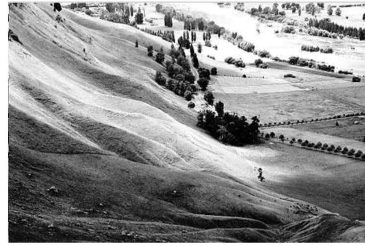
- 낮은 명암 대비(1번 그림) : 히스토그램의 값이 일부분에 집중되므로 명암의 구분이 되지 않습니다.
- 높은 명암 대비(2번 그림) : 히스토그램의 값이 양끝에 집중되므로 영상이 두 지역으로 구분 됩니다.
- 두 경우 모두 연산을 통해 명암 밝기 분포를 재분배하여 일정한 분포를 가지도록 하고 이를 히스토그램 평활화 라고 합니다.

<히스토그램 평활화 (=Histogram Equalization)>

= Image의 명암 대비를 높여서 선명하게 만드는 기법



원영상



평활화된 영상

히스토그램 평활화의 네 단계

1. 입력영상의 히스토그램 생성
 - 명암값 j 의 빈도수 $hist[j]$ 를 계산
2. 각 명암값 i 에 대하여 0부터 i 까지의 빈도수의 누적값을 계산

$$sum[i] = \sum_{j=0}^i hist[j]$$

3. 단계 2에서 구한 누적값을 정규화

$$n[i] = sum[i] \times \frac{1}{N} \times 255 \quad (N = \text{전체 픽셀 수})$$

4. 입력영상에서 픽셀 값 i 를 정규화된 값 $n[i]$ 로 변환하여 결과 영상 생성

• 정규화

$$n[i] = sum[i] \times \frac{1}{N} \times 255 \quad (N = \text{전체 픽셀 수})$$

40	40	50	60	70
40	50	60	70	70
50	60	70	70	80
60	70	70	80	90
70	70	80	90	100



명암값	누적합 sum[i]	정규화 값 n[i]
40	3	30
50	6	61
60	10	102
70	19	193
80	22	224
90	24	244
100	25	255

원 영상

- 정규화 과정에서 $sum[i]$ 는 밝기가 i 이하인 pixel의 개수입니다. i 가 60이하라면, 60이하의 pixel의 개수는 10개이므로 $sum[i]$ 가 10입니다.
- 히스토그램이 좁은 영역에 분포하는 영상은 명암대비가 낮아서 영상이 선명하지 않고, 히스토그램이 넓은 영역에 골고루 퍼져 있으면 명암대비가 높아서 영상이 선명합니다. 낮은 명암대비를 가지는 영상은 히스토그램 평활화에 의하여 높은 명암대비를 갖는 영상으로 변환되어 선명해집니다.

3) Thresholding

임계² 臨界 [임계/임계]  

1. 명사 사물이 어떠한 기준에 의하여 분간되는 한계.
2. 명사 물리 어떠한 물리 현상이 갈라져서 다르게 나타나기 시작하는 경계.

- = 밝기에 대해서 임계값을 설정하고 , 임계값을 기준으로 Image를 분할시킵니다.
- Single thresholding : 임계값 1개로 image를 양분시킵니다.
- Multiple thresholding : 임계값 2개로 image를 양분시킵니다.
- Adaptive Thresholding : 각 영상 데이터를 분석하여 최적화된 임계값을 찾아내서 분할시킵니다.
- 문제점 : 임계값에 따라 그림의 정확도가 많이 달라집니다.

<Example : T=128일 때의 이진 Thresholding>

$$P'(x, y) = \begin{cases} 255 & P(x, y) \geq T \\ 0 & P(x, y) < T \end{cases}$$



- 밝기가 128보다 크면 흰색 , 밝기가 128보다 작으면 모두 검은색으로 변경시킵니다.

4. 고급 알고리즘

1) 영상의 비트 플레인 분할

- Pixel은 1byte 이므로 8개의 bit로 이루어져 있으므로 총 8개의 bit plane으로 분할할 수 있습니다.
- 비트플레인 7 : 밝기 값이 128이상
- 비트플레인 0 : 보안 목적



(a) 원영상



(b) 비트 플레인 7



(c) 비트 플레인 6



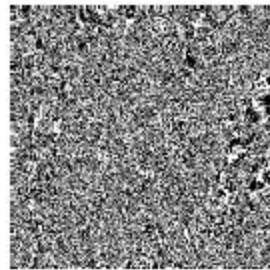
(d) 비트 플레인 5



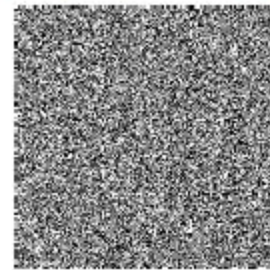
(e) 비트 플레인 4



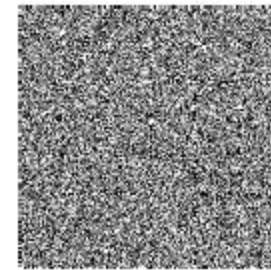
(f) 비트 플레인 3



(g) 비트 플레인 2



(h) 비트 플레인 1



(i) 비트 플레인 0

2) Otsu Thresholding

- Thresholding의 고급 기법
- 임계값을 컴퓨터가 알아서 적응적으로 계산해주는 기법.
- 임계값을 구하는 원리는 분산과 어려운 수학 공식으로 구해집니다.

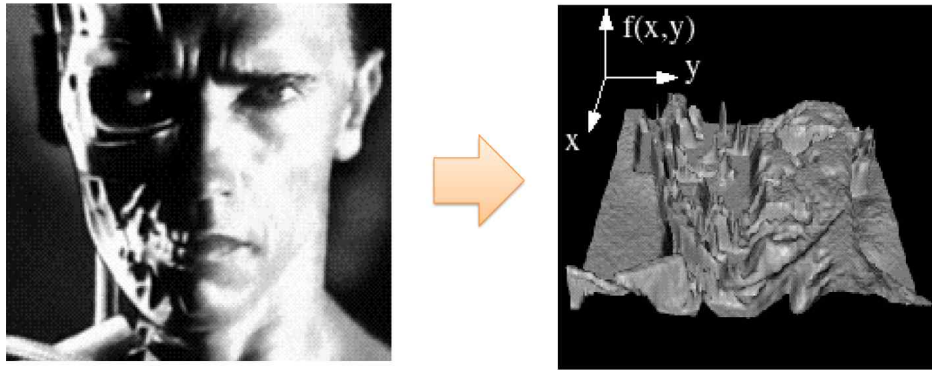
3) CLAHE(Contrast Limited Adaptive Histogram Equalization)

- 고급 히스토그램 평활화
- 기존의 히스토그램 평활화는 밝은 부분의 식별이 어렵습니다. 하지만 CLAHE를 사용하면 영상을 분할하여 각각 히스토그램 평활화를 수행합니다.

<2주차 : Convolution 과 Filtering>

*** 수식보다는 개념을 위주로 공부합니다. ***

1. 디지털 영상 = 함수

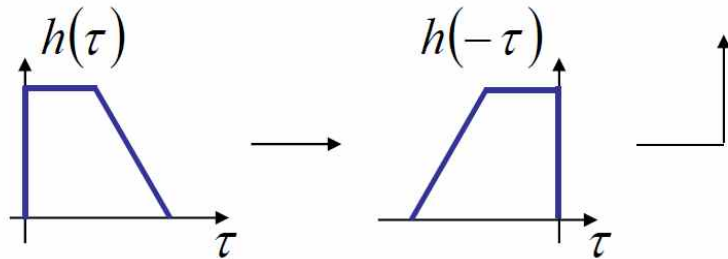


$$g(x, y) = t(f(x, y))$$

- 디지털 영상을 함수로 생각이 가능합니다.
- 디지털 영상처리는 입력 영상 f 를 변환 t 를 통하여 결과 영상 g 로 만들어내는 과정입니다.
- 2차원의 경우 (x,y) 좌표에 해당하는 값은 밝기를 나타내며 이를 함수로 나타냅니다.

2. Convolution

$$g(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)d\tau \quad g = f * h$$



- 연산기호 : *
 - T같이 생긴 기호를 타우라고 합니다. x축을 타우로 놓고 적분한 식을 해석해 보겠습니다.
 - 두 함수 f와 h를 그래프 상에 표현하고 함수 f는 움직이지 않고 고정시킵니다. 함수 h는 x값에 따라 이동시키고 대칭시켜 $-\infty \sim \infty$ 만큼 이동시킵니다.
 - 함수 h를 이동시키다보면 함수 f와 겹치는 부분이 나타나는데, 겹치는 부분의 면적을 Convolution이라고 합니다.
 - 적분 공간이 무한대(연속 공간)이므로 컴퓨터에서 계산이 불가능하므로, 유한 공간(이산 공간)으로 변화시켜서 구합니다.
 - 많이 겹칠수록 g의 값이 커지고, 적게 겹칠수록 g의 값이 0에 가까워집니다.
 - linear & shift invariant를 만족합니다.
 - convolution에 대해 교환법칙이 성립합니다.
- ex) $g(2)$: 고정된 함수 f와 x축에서 2만큼 이동시키고 대칭시킨 함수 h와 그래프에서 겹친 부분의 넓이

<Convolution을 불과 연기로 설명>

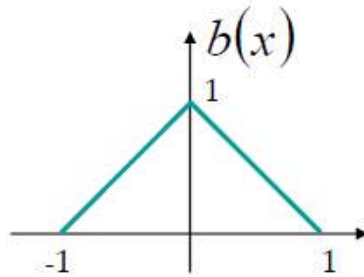
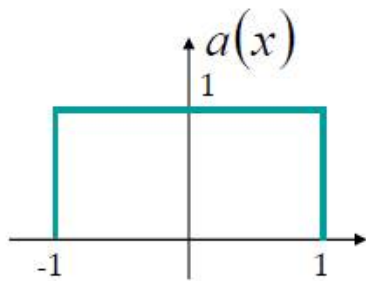
$$f * s = \sum_{i=0}^5 f(i) \cdot S(5 - i)$$

$$\int_0^t f(\tau) S(t - \tau) d\tau$$

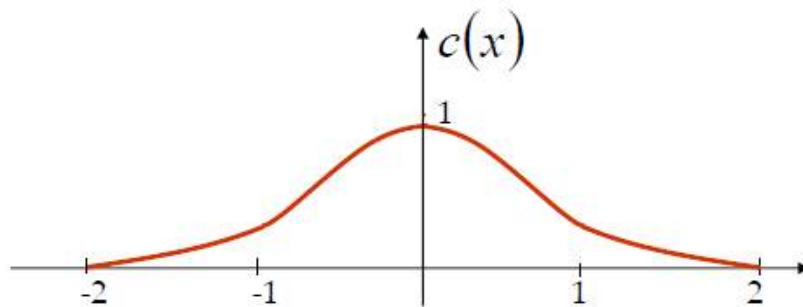
Question : 1분마다 성냥으로 불을 피웁니다. 불이 피울 때 마다 연기가 발생하게 되는데 5분 뒤에 공기 중에 연기의 양을 구하시오.

- Function_s = 공기 중의 성냥으로 인한 연기의 양
- Function_f = 분마다 피는 불의 개수
- 0분에 불을 1개 만들면 f(1)이고 , 57분에 불을 2개 만들면 f(2)입니다.
- s(값)에서 값은 연기가 공기 중에 존재한 시간
- 4시 0분에 불 1개 펴 -> 연기는 $1 * s(0)$
- 4시 1분에 불 2개 펴 -> 연기는 $1 * s(1) + 2 * s(0)$
- 4시 2분에 불 4개 펴 -> 연기는 $1 * s(2) + 2 * s(1) + 4 * s(0)$
- 4시 2분에 구한 식을 해석하면 , $1 * s(2)$ 에서 1은 4시 0분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 2분이 지났습니다. $2 * s(1)$ 에서 2는 4시 1분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 1분이 지났습니다. $4 * s(0)$ 에서 4는 4시 2분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 0분이 지났습니다.
- 5분 뒤에 연기의 양은 좌측 그림의 식이고 , 이걸 이산적으로 생각하지 않고 연속적으로 생각하면 우측 그림의 식이 되며 적분을 이용합니다.

Q&A : 컨벌루션의 결과는?



$$\downarrow c = a * b$$

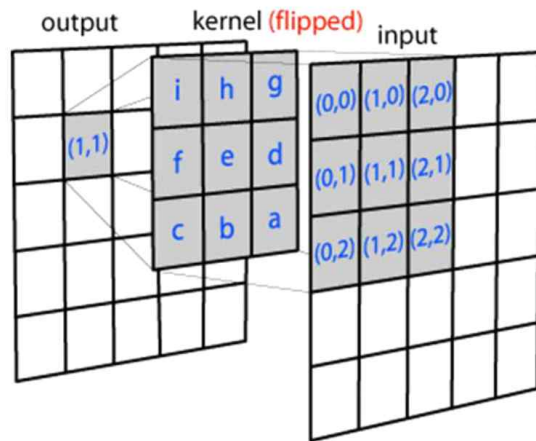


3. Convolution을 이용한 Filtering

- Filtering : 입력 영상에 filter mask를 convolution 해서 나오는 결과 영상
- Input Image에 filter mask를 입혀 연산을 하여 새로운 Output Image를 만드는데 이걸 변환(Transformation)으로 생각해도 괜찮은 것 같습니다. 그러므로 어떤 벡터에 변환을 가할 때 다양한 변환 행렬에 따라 다양한 결과 벡터가 출력되는 것처럼 다양한 filter mask를 입혀 filtering을 하면 다양한 Output Image가 출력됩니다.
- 동의어 : Filter Mask = Convolution Mask = Convolution Filtering Mask = Mask = Filter = kernel(filter의 하위 개념)
- 저는 Filter Mask로 사용하겠습니다.

1) Filtering Overview

<그림>



- Filter Mask가 (3x3)입니다.
- Output의 (1,1)을 구하기 위해 input의 (1,1)에 Mask의 중점을 올리고 겹치는 부분에 대해서 convolution을 진행합니다.
- 연산을 할 때 Mask는 Flip 시킵니다.
- 그 후 (Input의 0,0의 값 * i) + (input의 1,0의 값 * h) + etc + (Input의 2,2의 값 * a)를 구합니다.
- 이 계산 과정을 Convolution이라고 합니다,
- 예제 연습해서 맞추었습니다!!!

2) Filter Mask 특징

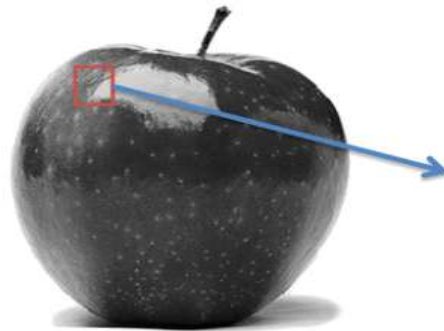
- Filter Mask의 크기는 홀수(3x3, 5x5, 7x7)
- Filter Mask의 모든 원소 합이 1이 되도록 설계하면 필터링 결과 영상의 평균 밝기가 입력 영상 평균 밝기와 같게 유지됩니다.
- Filter Mask의 모든 원소 합이 1 보다 작을 경우 영상의 평균 밝기가 어두워지고 1보다 크면 좀 더 밝아집니다.
- Filter Mask의 연산 결과 값이 음수일 수 있습니다. 음수라면 0으로 만들어 버리거나, 전체 값에 일정한 상수를 더하는 방식을 이용합니다.
- 1D Filter Mask로 2번을 연산하는 것($O(2n)$)이 2D Filter Mask 1번으로 연산하는 것($O(n^2)$) 보다 훨씬 빠릅니다.

3) Filter Mask를 input 영상과 겹쳤을 때 겹치지 않는 부분 해결책

- ① Filter Mask에서 input과 겹치지 않는 Filter Mask에서 h값은 0으로 합니다. 하지만 입력 영상 정보의 왜곡이 발생합니다.
- ② Filter Mask가 최대한 겹치도록 Filter Mask를 이동시킵니다. 하지만 Output 영상의 크기가 작아집니다.
- ③ Input 영상의 크기를 감소시킵니다.

4) Filtering 예시

- 흑백 image의 Input, Output에는 밝기 값이 들어갑니다.



Gray-scale Image

206	225	193	185	182	174	151
230	194	191	197	189	160	164
195	188	213	185	148	178	153
183	191	190	170	177	167	148
194	177	189	184	171	139	179
177	171	170	175	161	165	172
180	177	157	156	168	171	137
185	160	171	149	157	142	132
---	---	---	---	---	---	---

-1	-1	-1
-1	9	-1
-1	-1	-1

- Input의 (1,1)의 밝기 값은 194입니다. 거기에 위의 Filter를 이용해서 convolution을 해봅시다.
- 121로 밝기가 변해서 더욱 어두워집니다.

4. 다양한 필터링

- 1) 다양한 필터링이 필요한 이유 : 2가지 종류의 잡음(Noise)
= 잡음마다 다른 필터를 사용해서 효과적으로 잡음을 제거합니다.



Gaussian noise



Impulse noise

① 가우시안 잡음

- = 표준 분포를 따르는 잡음이며 , 영상의 다른 Pixel의 밝기 값과 비교하면 불규칙적으로 벗어나긴 하지만 뚜렷하게 다르지는 않은 Pixel 밝기 값.
- 평균 필터링은 가우시안 잡음을 없애는데 효과가 뛰어납니다.

② 임펄스 잡음

- = 랜덤한 분포를 따르는 잡음이며 , 영상의 다른 Pixel의 밝기 값과 비교하면 뚜렷하게 다른 Pixel의 밝기 값.
- 미디언 필터링은 미디언 필터링 설명 그림에서의 '밝기 값이 255인 pixel'같이 뚜렷하게 다른 밝기 값이 정렬 과정에서 삭제가 되므로 임펄스 잡음을 없애는데 효과가 뛰어납니다.

2) 평균 필터링 : 가우시안 잡음 해결책



입력 영상



3 X 3 마스크 이용



5 X 5 마스크 이용

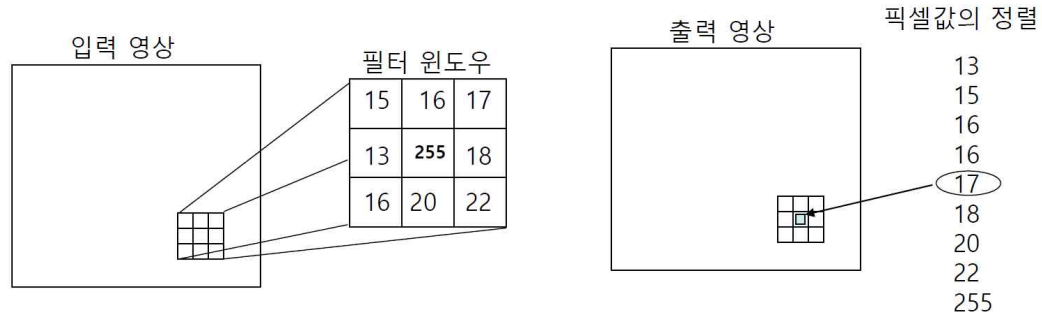
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$
$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$
$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$
$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$
$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$	$\frac{1}{25}$

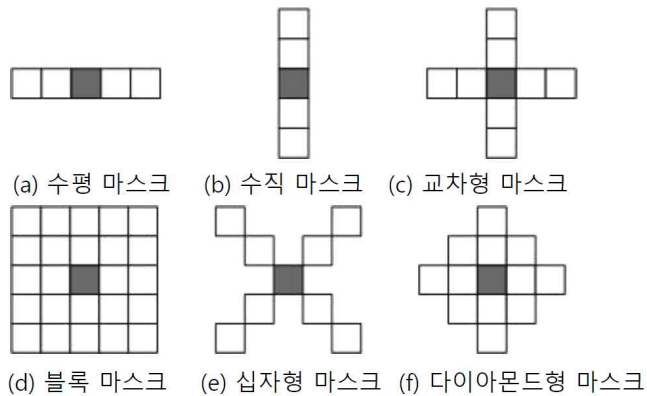
- 좌측은 3x3에서의 평균 필터링 Mask
- 우측은 5x5에서의 평균 필터링 Mask
- Mask의 각각의 index 값은 1/원소 개수
- 직사각형 형태의 Mask

- 평균 필터링을 하면 밝기 값의 차이가 줄어들기 때문에 Blurring(흐릿함)이 발생합니다.
- Mask의 크기가 커지면 Blurring의 정도가 더욱 커집니다. 왜냐하면 더 많은 영역에 대해서 평균치를 계산하기 때문에 자신의 밝기보다는 주변 pixel들의 밝기 값에 영향을 더 많이 받기 때문입니다.
- 경계를 blurring 합니다.
- Outlier에 민감합니다.

3) Median Filtering (=중간 값 필터링) : 임펄스 잡음 해결책



- 밝기 값이 255로 뭉친 구간이 아닌 이상 255는 정렬된 결과인 output의 중간 값이 될 수가 없으므로 Image 전체에서 밝기 값이 255인 pixel은 다른 밝기 값으로 변경될 것입니다.



= 기존에는 Filter에 특정 값을 넣어서 합이 0 OR 1이 되게 만들었지만 , Median Filtering에서는 Input Image에서 특정 pixel과 Filter Mask의 모양에 해당하는 주변 pixel들을 그대로 Filter Mask에 복사합니다. 그 후 Filter Mask에 저장된 밝기 값들을 정렬하고 중간 값을 밝기 값으로 하여 Output으로 출력합니다.

- 비선형 특성이므로 Convolution으로 구현이 불가능합니다. 그래서 위의 설명(Median Filtering의 방식)에서도 Convolution 과정이 존재하지 않습니다.
- Filter의 모양이 다양합니다.

4) 최소 / 최대 필터

- 교수님 설명 20초.

- 중간값 필터와 유사하나 중심 화소를 이웃 화소들의 중간 값으로 치환하는 대신 최소 또는 최대 값으로 치환
- 단극 임펄스 잡음 제거

5) Sharpen Filter (선명하게)

= Image를 선명하게 하는 기법

- Image의 Edge 성분을 모두 더합니다.

0	-1	0
-1	5	-1
0	-1	0

마스크 1

-1	-1	-1
-1	9	-1
-1	-1	-1

마스크 2

10	10	10	10	10	10	10	10	10
10	10	10	10	10	10	10	10	10
10	10	10	10	20	10	10	10	10
10	10	10	20	30	20	10	10	10
10	10	20	30	40	30	20	10	10
10	10	10	20	30	20	10	10	10
10	10	10	10	20	10	10	10	10
10	10	10	10	10	10	10	10	10
10	10	10	10	10	10	10	10	10



0	10	10	10	10	10	10	10	10
10	10	10	10	0	10	10	10	10
10	10	10	10	40	10	10	10	10
10	10	0	20	50	20	0	10	10
10	0	40	50	80	50	40	0	10
10	10	0	20	50	20	0	10	10
10	10	10	10	40	10	10	10	10
10	10	10	10	0	10	10	10	10
10	10	10	10	10	10	10	10	10

- 마스크 1을 Filter로 하여 Input에 Filtering을 한 결과입니다.
- Output Image에서 십자가 부분의 밝기가 증가하여 Image의 선명도가 증가했습니다.

5. 선형 평활화 기법 : 가우시안 평활화 (좀 더 자세하게 공부)

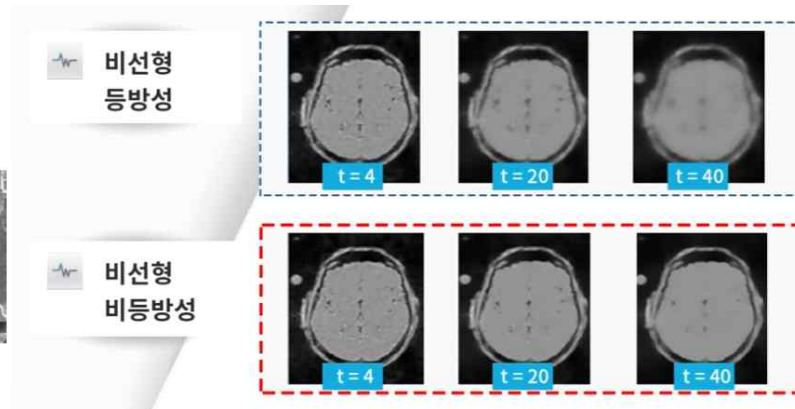
- = 선형 평활화 기법은 모든 Pixel들에 대해서 동일한 Convolution Mask를 적용하는 기법입니다.
- 1차원 convolution을 2번 적용하는 것이 2차원 convolution을 1번 적용하는 것 보다 빠릅니다.
- 원형 mask를 사용해서 사각형 mask를 사용하는 평균 필터링 보다 성능이 좋습니다.
- 하지만 영상에 존재하는 물체 간에 경계가 모호해지며 경계가 뭉개집니다.

6. 비선형 평활화 기법

- = 비선형 평활화 기법은 각 Pixel들에 대해서 주변 밝기 분포에 따라서 Convolution Mask를 적용하는 기법입니다. 영상의 물체에 대한 평활화는 선형 평활화 기법처럼 수행하지만 , 물체의 경계는 평활화를 조금만 수행시킵니다. 그 결과 영상에 존재하는 물체 간에 경계가 뚜렷해집니다.
- 기존의 가우시안 평활화에서 시그마 값을 50으로 올려버리면 영상이 흐릿해지는 문제가 발생합니다. 그 결과 영상의 물체와 배경의 경계가 모호해 집니다. 가우시안 평활화는 영상의 모든 Pixel에 동일한 Convolution Mask를 적용하기 때문에 이런 문제가 발생하므로 , 경계를 보존하는 비선형 평활화 기법을 사용합니다.
- 경계는 영상에서 밝기 값이 급격히 변하는 곳이므로 , Perona & Malik의 g 함수를 통해 구할 수 있습니다.

1) (비선형) 비등방성 확산 필터링

- = 열이 뜨거운 부분에서 차가운 부분으로 이동하는 성질을 밝기에 접목시켜 고안한 방식입니다. 영상의 밝은 부분을 뜨거운 부분으로 , 어두운 부분을 차가운 부분으로 생각하여 **영상의 밝은 부분에서 어두운 부분으로 밝기가 확산**되는 방법입니다.
- 비등방성은 방향에 관계가 있는 것을 의미합니다.
- (비선형) 등방성 확산 필터링도 (비선형) 평활화 기법에 포함되지만 비선형 비등방성 확산 필터링이 훨씬 선명하므로 고려하지 않습니다.



2) 양방향 필터링

- Convolution Mask의 형태를 영상의 내용에 따라 변화시킵니다.

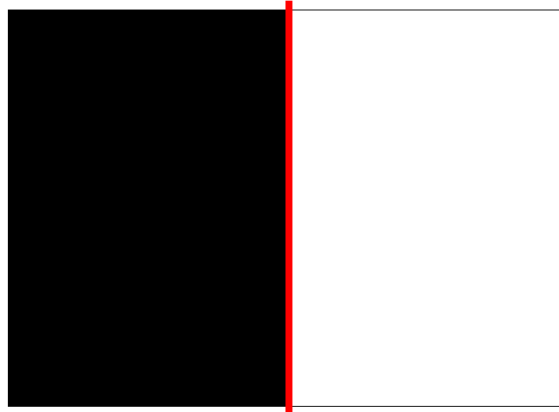
<3주차>

*** 컴퓨터 비전에서 사용하는 S/W를 간략하게 강의하신 주차입니다. PDF를 읽는 것으로 충분합니다. ***

*** 3주차 강의 내용은 시험 범위에 포함되지 않습니다. ***

<4주차 : 경계 검출>

1. 경계의 정의



0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255

0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255

= 객체의 외곽선(=Boundary)의 집합

- 전체 영상의 Pixel 비율로 따지면 매우 적습니다.

- 데이터는 이산적이고 잡음이 존재합니다.

- 좌측 그림 : 실세계 image를 보고 인간은 빨간 선이 경계인 것을 쉽게 판단할 수 있지만 , 컴퓨터는 실세계 image를 이용해서 판단하지 않습니다.

- 우측 그림 : Computer Vision에서 실세계 image를 나타낸 image입니다. 맨 첫 열을 1열로 할 때 , 컴퓨터는 3열 OR 4열을 경계로 생각합니다. **컴퓨터가 판단하는 경계는 반드시 pixel에 존재합니다.**

2. 경계 검출 성능을 판단

1) 정확한 검출 : 경계가 아닌 점을 경계로 찾거나 경계인 점을 검출하지 못하는 확률을 최소화해야 합니다.

2) 정확한 위치 : 실제 경계의 중심을 찾아야 합니다.

3) 단일 Edge : 하나의 경계는 하나의 점으로 표현되어야 합니다.

3. 컴퓨터가 경계를 판단하는 방법 : 축에 대한 미분

*** 교수님께서 경계 검출을 위한 식을 강조하셨습니다. ***

1) y축 방향의 경계 검출

0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255

0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255
0	0	0	255	255

이산 도메인에서의 y-축 방향의 경계 검출을 위한 세 가지 계산 방법

$$\frac{\Delta I}{\Delta x} \approx I(x + \Delta x) - I(x)$$

$$\frac{\Delta I}{\Delta x} \approx I(x) - I(x - \Delta x)$$

$$\frac{\Delta I}{\Delta x} \approx I(x + \Delta x) - I(x - \Delta x)$$

= 이산 데이터 : x축 방향의 밝기 값의 차이가 큰 부분

= 연속 데이터 : x축 방향으로 미분

- 이산 도메인에서의 y축 방향의 경계 검출을 위한 세 가지 계산 방법은 수치적으로 근사적으로 계산한 방식입니다.

2) x축 방향의 경계 검출

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
255	255	255	255	255
255	255	255	255	255

OR

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
255	255	255	255	255
255	255	255	255	255

이산 도메인에서의 x-축 방향의 경계 검출을 위한 세 가지 계산 방법

$$\frac{\Delta I}{\Delta y} \approx I(y + \Delta y) - I(y)$$

$$\frac{\Delta I}{\Delta y} \approx I(y) - I(y - \Delta y)$$

$$\frac{\Delta I}{\Delta y} \approx I(y + \Delta y) - I(y - \Delta y)$$

= 이산 데이터 : y축 방향의 밝기 값의 차이가 큰 부분

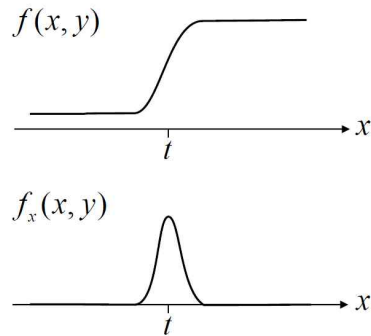
= 연속 데이터 : y축 방향으로 미분

3) 임의의 방향의 경계 검출 : Gradient(경사도) 이용

 $\nabla I = \left[\frac{\partial I}{\partial x}, 0 \right]$ 경계가 y축과 평행	 $\nabla I = \left[0, \frac{\partial I}{\partial y} \right]$ 경계가 x축과 평행	 $\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$ 경계와 수직방향
---	---	--

4. 다양한 경계 검출 알고리즘

1) 마스크를 이용한 경계 검출



- 위의 그래프는 pixel의 x 축 위치에 대해서 y 값으로 해당 pixel의 밝기 값을 나타낸 그래프입니다.
- 아래의 그래프는 위에서 구한 밝기 그래프를 미분한 그래프입니다.
- 그러므로 미분한 그래프의 y 값이 밝기의 변화율 이므로 , 변화율이 큰 t 지점이 경계입니다.

<3가지 경계 검출 마스크>

0	-1
1	0

-1	0	1
-1	0	1
-1	0	1

-1	0	1
-2	0	2
-1	0	1

-1	0
0	1

-1	-1	-1
0	0	0
1	1	1

-1	-2	-1
0	0	0
1	2	1

(a) 로버츠

(b) 프리윗

(c) 소벨

- 3가지 경계 검출 마스크 모두 2가지 씩 존재합니다.
- 서로 방향이 다른 마스크 2가지를 각각의 경계 검출 마스크가 갖고 있습니다.
- ① 로버츠 : 두 마스크는 대각선의 방향이 다릅니다.
- ② 프리윗 : 두 마스크는 x 축 , y 축 기준으로 방향을 갖습니다. 중점을 기준으로 근처 pixel이 1의 가중치를 갖습니다.
- ③ 소벨 : 두 마스크는 x 축 , y 축 기준으로 방향을 갖습니다. 중점을 기준으로 근처 pixel이 2의 가중치를 갖습니다. 소벨이 중점 부근의 pixel 값을 더 많이 봅니다. 프리윗과 성능적으로 큰 차이는 없지만 **소벨 마스크를 경계 검출 마스크로 가장 많이 사용합니다.**
- 프리윗과 소벨이 로버츠 보다 경계 검출 성능이 더 뛰어납니다.

2) 그래디언트 크기를 이용한 경계 검출

- 그래디언트는 벡터입니다.

- 로버츠 필터를 이용한 그래디언트 크기 계산
- 프리윗 필터를 이용한 그래디언트 크기 계산
- 소벨 필터를 이용한 그래디언트 크기 계산

$$\sqrt{\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}^2 + \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}^2} \quad \sqrt{\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}^2 + \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}^2} \quad \sqrt{\begin{bmatrix} 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}^2 + \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}^2}$$

3) 캐니 경계 검출 알고리즘 : 시그마 값을 변화 시키면서 경계 검출의 정도를 변화시킵니다.



① 가우시안 평활화 필터링
= 영상 부드럽게 만들기

② 그래디언트 계산
= 소벨 마스크를 사용합니다.

③ 비 최대(Non-Maximum) 억제
= 하나의 경계가 여러 개의 Pixel로 표현되는 현상을 없애기 위해 그래디언트 크기가 최대인 pixel만을 경계로 설정합니다.

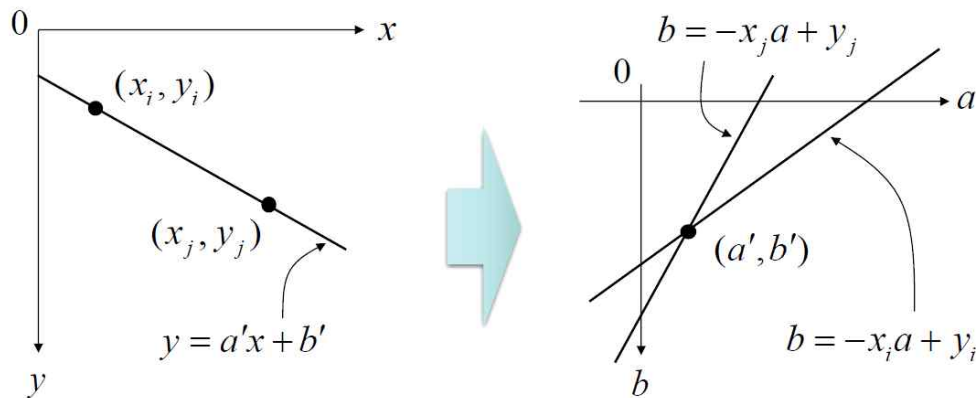
④ 히스테리시스 경계 트래킹
= 두 임계값(T_Low와 T_High)을 두어 그래디언트 크기가 T_High보다 크면 강한 경계 & 최종 경계로 선정하고, T_Low ~ T_High이면 강한 경계와 연결 되어 있는 pixel만 최종 경계로 선정합니다.

4) 허프 변환을 이용한 직선 검출 알고리즘

허프 변환(Hough transform)

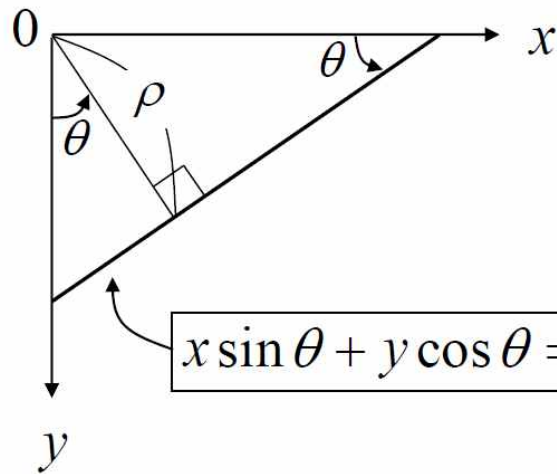
- 2차원 영상 좌표에서의 직선의 방정식을 파라미터(parameter) 공간으로 변환하여 직선을 찾는 알고리즘

$$y = ax + b \quad \Leftrightarrow \quad b = -xa + y$$



직선의 방정식 $y = ax + b$ 를 사용할 때의
문제점

- y 축과 평행한 수직선을 표현하지 못함
→ 극좌표계 형태의 직선의 방정식을 사용



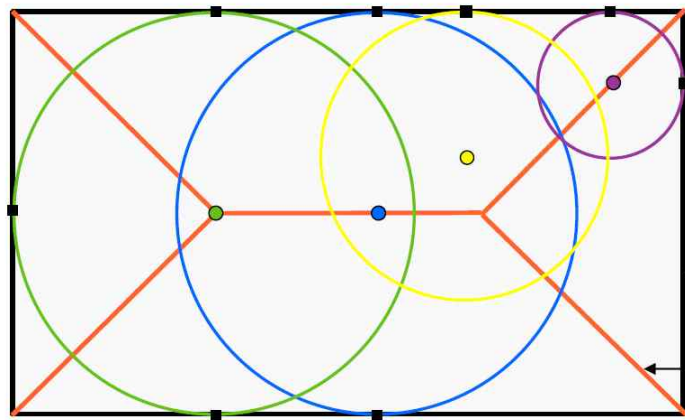
$$x \sin \theta + y \cos \theta = \rho$$

$$-\sqrt{M^2 + N^2} \leq \rho \leq \sqrt{M^2 + N^2}$$
$$0 \leq \theta \leq \pi$$

<5주차 : Thinning(=세선화)>

1. Medial Axis (축)

- = 두 개 이상의 경계로부터 다수의 가장 가까운 위치를 갖는 내부 점들의 집합
- 매우 얇음
- 물체의 형태를 보존합니다.
- 경계가 실세계에서 매끄럽지 않습니다.
- skeleton은 medial axis의 근사입니다.



← 물체

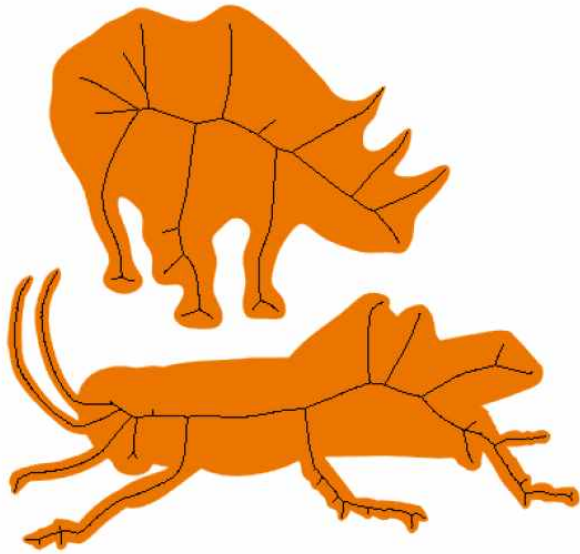
← 미디얼 축

- 주황색 선이 medial axis

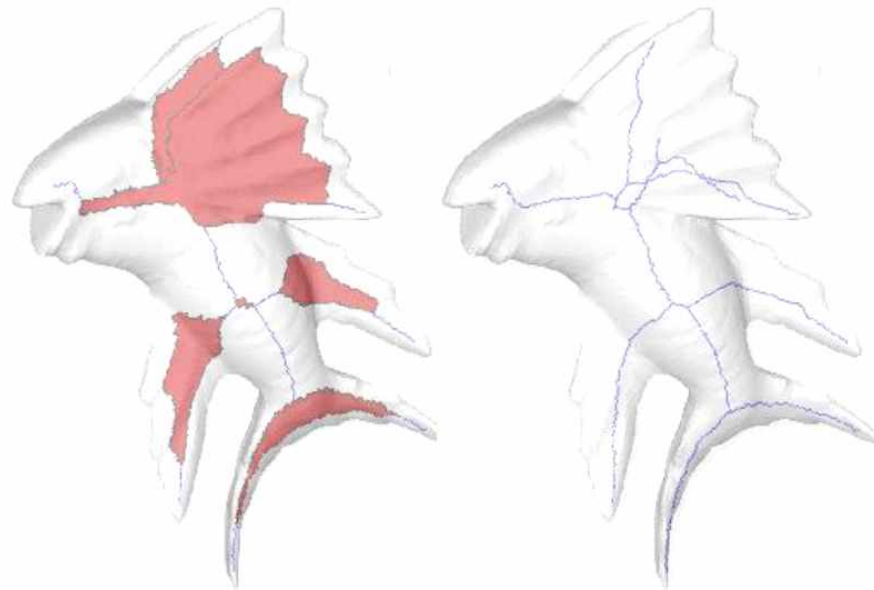
2. Skeleton

= 물체의 중심 부위의 geometry(기하학적 구조)

- 컴퓨터 그래픽스에 매우 많이 사용 됩니다.
- 수학적인 medial Axis를 근사하여 실용으로 만든 것이 Skeleton입니다.
- 경계가 매끄럽지 않을 경우에도 비교적 정확하게 추출됩니다.



2차원 형태의 스켈렉턴 :
1차원 곡선

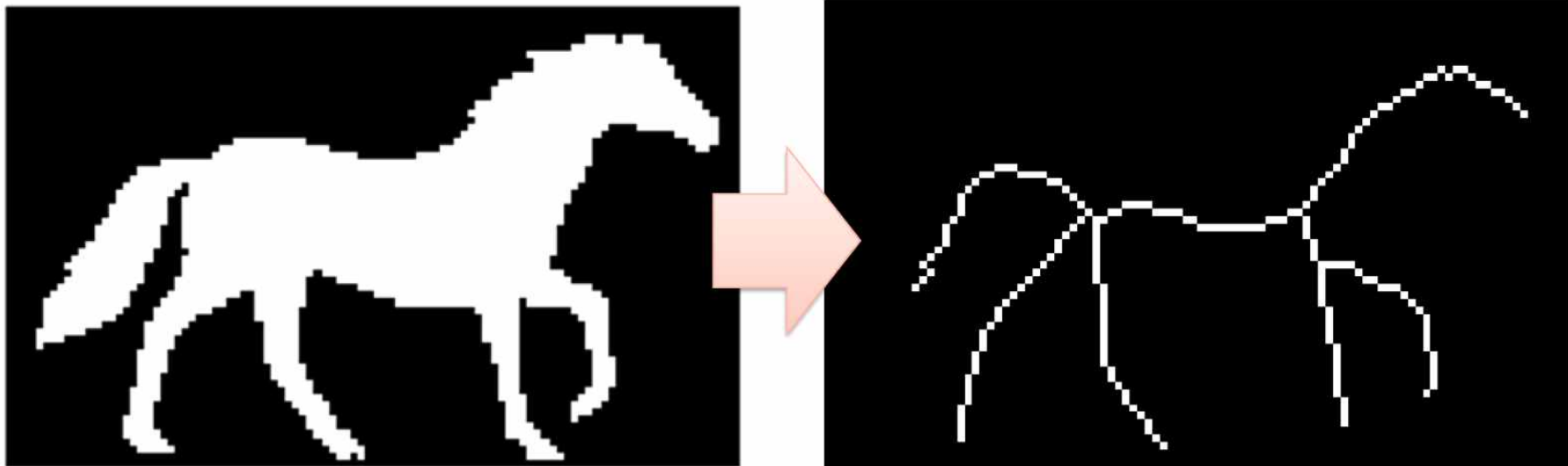


3차원 형태의 스켈렉턴 :
1차원 곡선과 2차원 표면

3. Skeleton을 구현하는 방법 : 2D Thinning

= 물체와 배경의 영상으로부터 물체의 가장 외곽에 존재하는 경계로부터 한 pixel 두께로 특정 Pixel들을 반복적으로 제거할 때 남는 최종 pixel의 집합입니다.

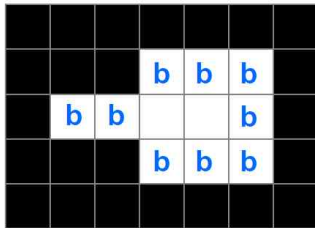
- 2D Thinning의 결과 , Image에서 물체는 최종적으로 뼈대만 남습니다.
- 제거 하는 특정 Pixel : (Border pixel과 Simple pixel을 제거하고 , Curve-End Pixel을 제거하지 않습니다.)
- Image는 물체 pixel과 배경 pixel로 나눌 수 있습니다.



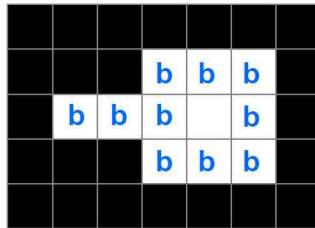
1) 2D Thinning에서 정의한 Pixel

① Border pixel : 물체 내부에서 **배경에 인접한 pixel**

- 4 – 연결성 : 대각선 연결 불포함
- 8 – 연결성 : 대각선 연결 포함

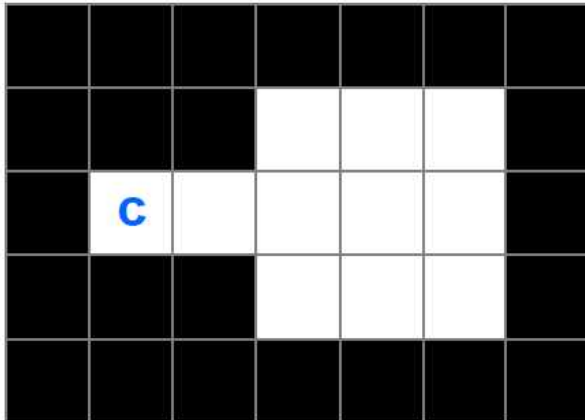


Border pixels (4-연결성)



Border pixels (8-연결성)

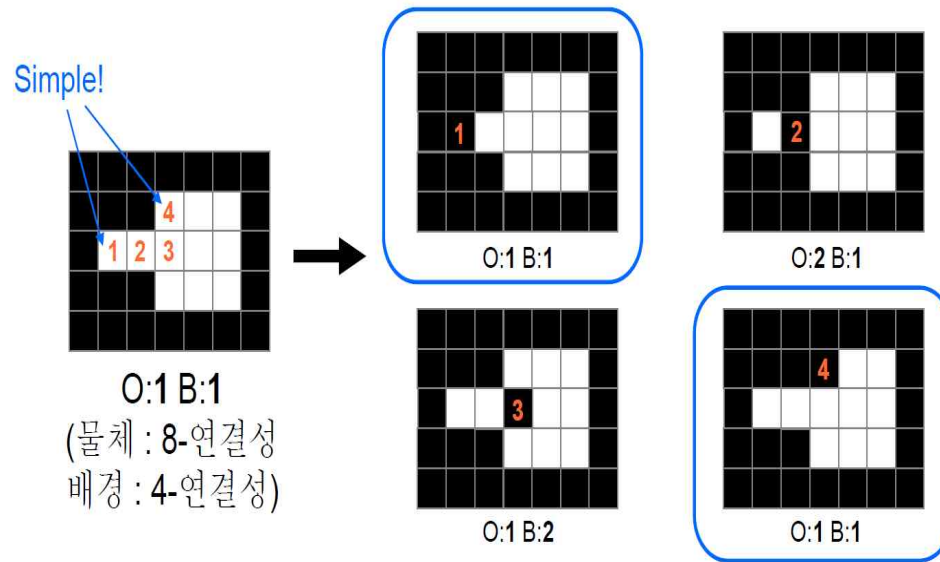
② Curve-End Pixel : **단 하나의 물체 pixel과 연결된 pixel**



③ Simple Pixel

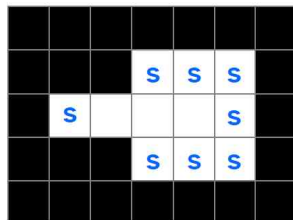
<Simple Pixel의 조건 두 가지>

- 물체 pixel 중에서 제거 했을 때 , 물체와 배경의 개수가 동일합니다.
- 물체 pixel 중에서 제거 했을 때 , 물체에서 구멍이 생기지 않습니다.



<Question : 4개의 pixel 중 Simple Pixel을 고르시오.>

- ① : 제거해도 위상은 변화하지 않기 때문에 O
- ② : 제거하면 물체가 2개로 나누어지기 때문에 X
- ③ : 물체에 구멍이 생기기 때문에 X
- ④ : 제거해도 물체와 배경의 개수가 동일하기 때문에 X

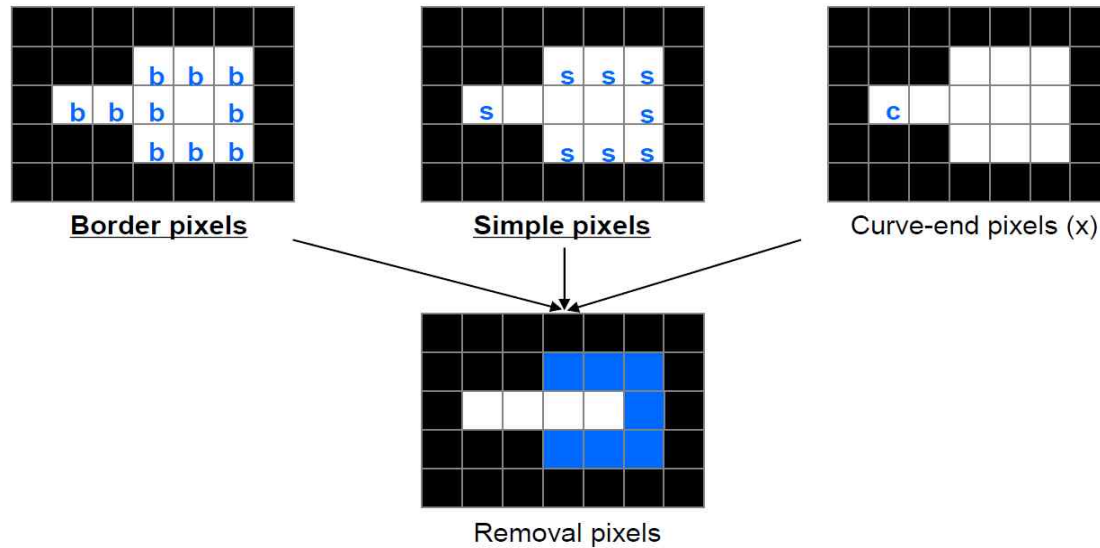


- S는 Simple pixel이며 , 이들을 제거해도 물체의 Topology가 변하지 않습니다.
- 다시 말해 , Simple Pixel은 제거하여도 물체와 배경의 개수가 동일합니다.

2) Thinning 과정에서 제거 가능한 pixel

= 어떤 Pixel이 border pixel 이고 , simple pixel 이지만 , curve-end pixel은 아닌 Pixel이면 제거가 가능합니다.

Border 픽셀 & Simple 픽셀 & Curve-end X



- 물체의 가장 왼쪽에 존재하는 Pixel은 border pixel 이고 , simple pixel이지만 , 동시에 curve-end pixel이므로 제거할 수 없습니다.

3) Thinning의 2가지 제거 순서 알고리즘

① Parallel Thinning

= 제거 가능한 모든 pixel을 **동시에** 제거합니다.

② Serial Thinning

= 제거 가능한 pixel을 **순차적**으로 방문하여 제거하기 전 simple 특성을 다시 한 번 체크하고 제거합니다.

- 1개씩 제거하기 때문에 제거하는 pixel의 **방문 순서에 따라서 Skeleton의 모양이 달라질 수 있습니다.**

4. Skeleton Pruning

= Noise로 발생하는 의미 없는 skeleton을 제거해 주는 방법

- Thinning 은 경계 잡음에 매우 민감함
 - 미디얼 축 개념의 근본 한계점
- 스켈렉톤 프루닝
 - Thinning 과정 중
 - End pixels (voxels)에 대한 더 선택적인 조건을 부여함
 - Thinning 처리 후
 - Branch 길이에 따른 제거



경계 잡음



결과 스켈렉톤

<6주차_1 : 특징 검출을 이해하기 위한 행렬 지식>

*** 강의 초반의 기초 행렬은 필기하지 않았습니다. ***

*** 선형대수학 4강의 내용이 포함되어 있습니다. 추후에 필요하면 4강에 옮기기 ***

1. 전치 행렬

A

a

-b

b

a

→

A^T

a

b

-b

a

전치행렬

8

1

6

8

3

4

→

3

5

7

1

5

9

4

9

2

6

7

2

$$(1) (A^t)^t = A$$

$$(2) (kA)^t = kA^t$$

$$(3) B가 m \times n \text{ 행렬이면 } (A+B)^t = A^t + B^t$$

$$(4) B가 n \times p \text{ 행렬이면 } (AB)^t = B^t A^t \text{ -----}$$

2. 소행렬식과 여인자

정의 [소행렬식과 여인자]

정사각행렬 $A = [a_{ij}]$ 의 i 행과 j 열을 제거하여 만든 부분행렬을 $A(i|j)$ 라 하고 그의 행렬식 $M_{ij} = \det A(i|j)$ 를 A 의 a_{ij} 에 대한 **소행렬식(minor)**이라 한다. 또, $A_{ij} = (-1)^{i+j} |A(i|j)| = (-1)^{i+j} M_{ij}$ 를 A 의 a_{ij} 에 대한 **여인자(cofactor)**라고 한다.

예제 1

행렬 $A = \begin{bmatrix} 3 & 1 & -4 \\ 2 & 5 & 6 \\ 1 & 4 & 8 \end{bmatrix}$ 이라 할 때, A 의 a_{11} 에 대한 소행렬식과 여인자를 구하여라.

풀이

A 의 a_{11} 에 대한 소행렬식은 $M_{11} = \det A(1|1) = \begin{vmatrix} 1 & -4 \\ 4 & 8 \end{vmatrix} = \begin{vmatrix} 5 & 6 \\ 4 & 8 \end{vmatrix} = 16$ 이고,

a_{11} 에 대한 여인자는 $A_{11} = (-1)^{1+1} M_{11} = 16$. ■

- 행렬식에서 중요한 계산으로 사용됩니다.
- 여인자는 소행렬식을 이용해서 계산됩니다.

3. determinant(행렬식)

= det

= 행렬을 대표하는 값

= 절댓값으로 표기합니다. $\det A = |A|$

ex) 2 X 2 행렬의 det : $ad-bc$

ex) 3 X 3 행렬의 det

$$= a_{11}a_{22}a_{33} - a_{11}a_{23}a_{32} + a_{12}a_{23}a_{31} \\ - a_{12}a_{21}a_{33} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31}$$

<3 X 3 행렬의 det을 쉽게 계산하는 방법 : 소행렬>

= n x n행렬 A에 대하여 A의 i행, j열을 제외하고 얻은 (n-1) x (n-1) 행렬식

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix} = a_{11} \begin{vmatrix} a_{22} & a_{23} \\ a_{32} & a_{33} \end{vmatrix} - a_{12} \begin{vmatrix} a_{21} & a_{23} \\ a_{31} & a_{33} \end{vmatrix} + a_{13} \begin{vmatrix} a_{21} & a_{22} \\ a_{31} & a_{32} \end{vmatrix}$$

4. 특수한 행렬의 det

1) 전치행렬

- 행렬 A 의 det 값과 , 행렬 A 의 전치행렬 A^T 의 det 값은 같습니다.

2) 행렬의 상수 배

정리 4.1.6

정사각행렬 A 의 한 행을 k 배하여 얻어진 행렬을 B 라 하면 $|B| = k|A|$ 이다.

예제 10

행렬 $A = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 5 & 3 \\ 2 & 8 & 6 \end{bmatrix}$ 의 행렬식을 계산하면 다음과 같다.

$$|A| = \begin{vmatrix} 1 & 2 & 3 \\ 1 & 5 & 3 \\ 2 & 8 & 6 \end{vmatrix} = 2 \begin{vmatrix} 1 & 2 & 3 \\ 1 & 5 & 3 \\ 1 & 4 & 3 \end{vmatrix} = (2)(3) \begin{vmatrix} 1 & 2 & 1 \\ 1 & 5 & 1 \\ 1 & 4 & 1 \end{vmatrix} = (2)(3)(0) = 0 \quad \blacksquare$$

3) 삼각행렬

정리 4.1.9

$A = [a_{ij}]$ 가 n 차의 삼각행렬이면 A 의 행렬식은 주대각선성분의 곱과 같다. 즉,

$$|A| = a_{11}a_{22} \cdots a_{nn}$$

예제 12

앞의 정리에 의하여 $\begin{vmatrix} 2 & 7 & -3 & 8 & 3 \\ 0 & -3 & 7 & 5 & 1 \\ 0 & 0 & 6 & 7 & 6 \\ 0 & 0 & 0 & 9 & 8 \\ 0 & 0 & 0 & 0 & 4 \end{vmatrix} = (2)(-3)(6)(9)(4) = -1296 \quad \blacksquare$

5. det의 성질



$\det A = 0 \rightarrow$ 역행렬이 존재하지 않음



$\det A \neq 0 \rightarrow$ 역행렬이 존재



행렬식은 정사각행렬에서만 정의됨
(역행렬도 동일함)

6. 선형 변환 (Linear Transformation)

정의

$\mathbb{R}^n$ 에서 $\mathbb{R}^m$ 으로의 변환 $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 가 임의의 벡터 $u, v \in \mathbb{R}^n$ 와 임의의 스칼라 k 에 대하여 다음 두 조건을 만족하면 T 를 $\mathbb{R}^n$ 에서 $\mathbb{R}^m$ 로의 **선형변환(linear transformation)**이라고 한다.

$$(1) \quad T(u+v) = T(u) + T(v) \qquad (2) \quad T(ku) = kT(u) \quad (k \in \mathbb{R})$$

특히, $\mathbb{R}^n$ 에서 $\mathbb{R}^n$ 자신으로의 선형변환 $T: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 를 $\mathbb{R}^n$ 위의 **선형연산자(linear operator)**라고 한다.

- linear transformation은 많은 성질을 만족하기 때문에 매우 중요한 개념입니다.

✉ 예제

$\mathbf{u}_1 = (1, 1, 0)$, $\mathbf{u}_2 = (1, 0, 1)$, $\mathbf{u}_3 = (0, 1, 1)$ 인 $\mathbb{R}^3$ 의 한 기저 $\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\}$ 가 있다. 이때 선형변환 $T: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ 을

$$T(\mathbf{u}_1) = (2, -3), \quad T(\mathbf{u}_2) = (-4, 6), \quad T(\mathbf{u}_3) = (-2, 4)$$

로 정의한다. $T((3, 5, -4))$ 를 구하여라.

$$\mathbf{u} = \alpha_1 \mathbf{u}_1 + \alpha_2 \mathbf{u}_2 + \alpha_3 \mathbf{u}_3$$

즉 $(3, 5, -4) = \alpha_1(1, 1, 0) + \alpha_2(1, 0, 1) + \alpha_3(0, 1, 1)$

에서

$$\begin{cases} \alpha_1 + \alpha_2 = 3 \\ \alpha_1 + \alpha_3 = 5 \\ \alpha_2 + \alpha_3 = -4 \end{cases}$$

$$\therefore \alpha_1 = 6, \quad \alpha_2 = -3, \quad \alpha_3 = -1$$

$$\therefore \mathbf{u} = (3, 5, -4) = 6\mathbf{u}_1 - 3\mathbf{u}_2 - \mathbf{u}_3$$

$$\therefore T(\mathbf{u}) = T((3, 5, -4)) = 6T(\mathbf{u}_1) - 3T(\mathbf{u}_2) - T(\mathbf{u}_3)$$

$$= 6(2, -3) - 3(-4, 6) - (-2, 4)$$

$$= (26, -40)$$



- 선형 변환의 2가지 정의를 이용하면 쉽게 해결할 수 있습니다.
- 식을 세워 3차 연립방정식을 해결하면 계수를 구할 수 있고, $6*\mathbf{u}_1 + -3*\mathbf{u}_2 + -1*\mathbf{u}_3$ 을 계산하면 됩니다.

7. 고유 값과 고유 벡터

정의 [고유값과 고유벡터]

A 를 n 차의 정사각행렬이라 하자. 0 아닌 벡터 $x \in \mathbb{R}^n$ 가 적당한 스칼라 λ 에 대하여 다음을 만족하면 λ 를 A 의 **고유값(eigenvalue)**이라 하고, x 를 λ 에 대응하는 A 의 **고유벡터(eigenvector)**라고 한다.

$$Ax = \lambda x$$

예제 1 만일 $A = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$, $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 라면 $Ax = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 3x$ 이다. 따라서 고유값은 3이고, $x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 는 3에 대응하는 고유벡터이다. ■

1) 고유 값 (eigenvalue)

= 람다로 표현하며, **상수(스칼라)**입니다.

<고유 값이 존재하기 위한 조건>

$$Ax = \lambda x = \lambda Ix, Ax - \lambda Ix = 0$$

$$(A - \lambda I)x = 0$$

$$\det(A - \lambda I) \neq 0 \rightarrow x = 0 \text{ 벡터}$$

$$\det(A - \lambda I) = 0 \rightarrow \text{고유값이 존재함}$$



A 가 $n \times n$ 행렬일 때 A 의 고유값이 λ 이기 위한 필요충분조건은

$$\det(A - \lambda I) = 0$$

2) 고유 벡터 (eigenvector)

= 선형 변환 A 에 의한 변환 결과가 자기 자신의 상수배가 되는 0 이 아닌 **벡터**

- 결국 고유 벡터는 고유 값이 정해져야 구할 수 있습니다.

<고유 값과 고유 벡터를 이해하기 위한 최고의 예제>

예제 3 행렬 $A = \begin{bmatrix} 5 & -6 \\ 2 & -2 \end{bmatrix}$ 의 고유값과 고유벡터를 모두 구하여라.

풀이

만일 벡터 $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ 에 대하여 $Ax = \lambda x$ 를 만족한다고 하자. 그러면

$$\begin{bmatrix} 5 & -6 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Leftrightarrow \begin{cases} 5x_1 - 6x_2 = \lambda x_1 \\ 2x_1 - 2x_2 = \lambda x_2 \end{cases} \Leftrightarrow \begin{cases} (\lambda - 5)x_1 + 6x_2 = 0 \\ -2x_1 + (\lambda + 2)x_2 = 0 \end{cases} \quad (1)$$

그런데 앞서 언급한 대로 이 동차연립방정식은 자명한 해(0)를 제외한 무수히 많은 해를 가져야 하므로

$$\begin{vmatrix} \lambda - 5 & 6 \\ -2 & \lambda + 2 \end{vmatrix} = 0 \Leftrightarrow \lambda^2 - 3\lambda + 2 = 0 \Leftrightarrow (\lambda - 1)(\lambda - 2) = 0$$

$$\therefore \lambda = 1, 2$$

① $\lambda_1 = 1$ 에 대응하는 고유벡터를 구해보자.

$$(1) \text{로부터 } \begin{cases} -4x_1 + 6x_2 = 0 \\ -2x_1 + 3x_2 = 0 \end{cases} \Leftrightarrow x_1 - \frac{3}{2}x_2 = 0$$

$$\therefore x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3s \\ 2s \end{bmatrix} = s \begin{bmatrix} 3 \\ 2 \end{bmatrix} \quad (s \in \mathbb{R} \setminus \{0\})$$

② $\lambda_2 = 2$ 에 대응하는 고유벡터를 구해보자.

$$(1) \text{로부터 } \begin{cases} -3x_1 + 6x_2 = 0 \\ -2x_1 + 4x_2 = 0 \end{cases} \Leftrightarrow x_1 - 2x_2 = 0$$

$$\therefore x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2t \\ t \end{bmatrix} = t \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad (t \in \mathbb{R} \setminus \{0\})$$

- $\det(A - \lambda I) = 0$ 을 이용하여 고유 값 λ 를 구합니다.

- 고유 값을 2개 구했고, 각각의 고유 값에 대해 연립 방정식을 계산하여 고유 벡터를 구합니다.

8. 대각화 가능 행렬 (D)

- = 행렬에서 대각선을 제외한 나머지 값이 모두 0인 행렬
- 어떤 행렬을 대각화 한 행렬로 바꾸면 그 행렬로 다른 계산이 매우 간편해 집니다.

<대각화 방법>

$$A = \begin{bmatrix} 2 & -1 & 4 \\ 0 & 1 & 4 \\ -3 & 3 & -1 \end{bmatrix}$$

1) 고유 값 구하기

- $\det(A - \lambda I) = 0$ 을 이용하여 고유 값 λ 를 구합니다.
- 주어진 행렬 A에 대해서 고유 값은 -1 , 1 , 2가 됩니다.

2) 각각의 고유 값에 대한 고유 벡터 구하기

$$(A + I)\mathbf{x}_1 = 0, \quad (A - I)\mathbf{x}_2 = 0, \quad (A - 2I)\mathbf{x}_3 = 0$$

$$\mathbf{x}_1 = \begin{bmatrix} 2 \\ 2 \\ -1 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix}$$

- 3) 고유 벡터 들을 나열에서 행렬 P를 만들고 , $P * A * P^{-1}$ 를 계산해서 대각화 가능 행렬 D를 구합니다.
 - P는 고유 벡터 들을 나열하면 됩니다.

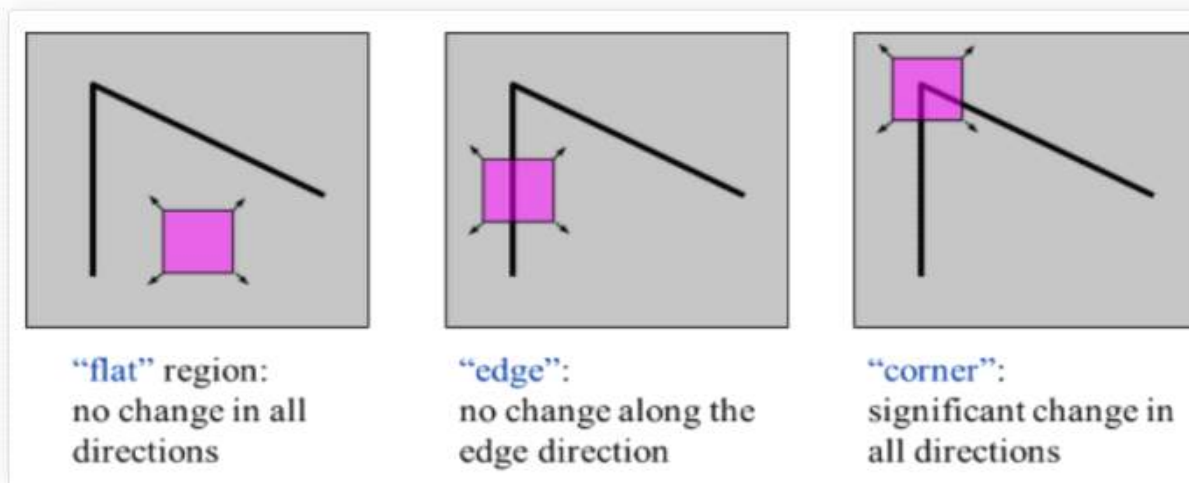
P와 P⁻¹ 계산

$$P = [\mathbf{x}_1 \mid \mathbf{x}_2 \mid \mathbf{x}_3] = \begin{bmatrix} 2 & 1 & 3 \\ 2 & 1 & 4 \\ -1 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} P^{-1}AP &= \begin{bmatrix} -1 & 1 & -1 \\ 6 & -5 & 2 \\ -1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 2 & -1 & 4 \\ 0 & 1 & 4 \\ -3 & 3 & -1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 3 \\ 2 & 1 & 4 \\ -1 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 2 \end{bmatrix} (=D) \end{aligned}$$

<6주차_2 : 특징 검출 기법>

1. 기초 용어



- 이번 강의에서는 corner 가 핵심.
- Corner가 Corner Point입니다.

2. 특징 (Feature)

- = 다른 pixel 들과 비교해서 **밝기가 두드러지는 점(vertex)**
- 영상 정렬에 활용됩니다.
- 3차원 재구성에 활용됩니다.
- 움직임 추적에 활용됩니다.
- 특징점 검출 -> 특징점 기술(벡터로 표현) -> 특징이 비슷한 점 끼리 매칭
- 두 가지 영상을 병합해서 하나로 만듭니다.

ex) Corner Point

ex) Panorama Stitching : 두 장의 영상을 병합하는 방법

3. 좋은 특징점의 조건

• 반복 재생성

- 같은 영상이라면, 기하학적/광학적 변환이 있다고 하더라도 동일한 특징점이 검출되어야 함

• 변별성

- 각 특징점들은 주변에 비해서 두드러져야 함

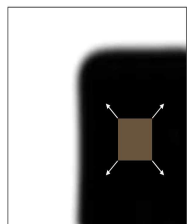
• 압축성과 효율성

- 전체 영상의 픽셀 숫자에 비해서 특징점들의 숫자가 적어야 함

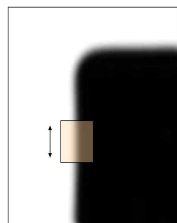
• 지역성

- 전체 영상에서 두드러지는 일부 영역에 분포해서 occlusion 등에 강인해야 함

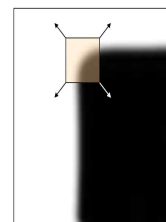
- Corner Point는 좋은 특징점의 조건을 모두 만족하기 때문에 특징점을 검출할 때 Corner Point를 찾습니다.
- Image에서 물체 내부의 pixel OR 배경의 pixel은 근처의 다른 pixel로 이동시켰을 때 밝기 값의 변화가 크지 않습니다.
- Image에서 경계는 경계와 수직방향으로 다른 pixel과 비교하면 밝기 값의 변화가 크지만, 수평방향으로 다른 pixel과 비교하면 밝기 값의 변화가 크지 않습니다.
- Image의 Corner Point는 어떤 방향으로 이동해도 다른 pixel과 비교하면 밝기 값의 변화가 크기 때문에 좋은 특징점 입니다.



물체 내부 or 배경
모든 방향으로 변화가 거의 없음

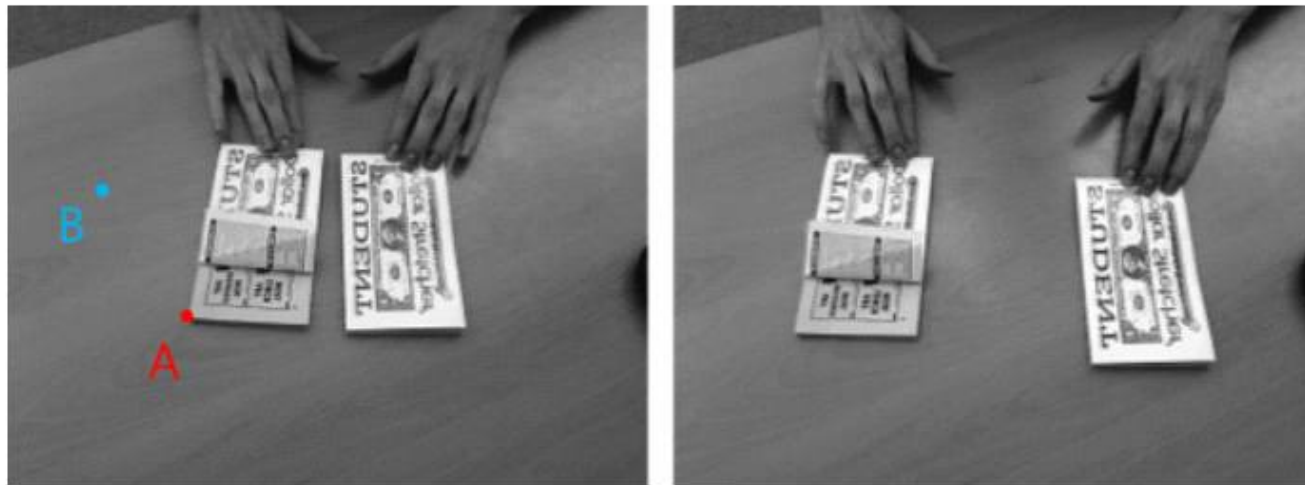


경계
경계 방향으로 변화가 거의 없음



코너 포인트
모든 방향으로 변화가 큼

영상에서 물체를 추적하거나 인식할 때, 영상과 영상을 매칭할 때 가장 일반적인 방법은 영상에서 주요 특징점(keypoint)을 뽑아서 매칭하는 것입니다. 특징점을 영어로는 보통 keypoint 또는 interesting point라 부릅니다.



<그림 1>

위 그림에서 첫번째 이미지와 대응되는 지점을 두번째 이미지에서 찾다고 했을 때, A는 쉽게 찾을 수 있지만 B는 찾기가 어렵습니다. 이와 같이 영상을 매칭하는데 있어서 A처럼 주위 배경과 구분되면서 식별이 용이한 지점을 특징점으로 잡는게 유리함은 누구나 쉽게 알 수 있을 것입니다.

4. 특징점 검출 방법 : 코너 검출

*** 엄밀히 말하면 가장 좋은 특징점인 Corner Point를 검출 하는 방법입니다. ***

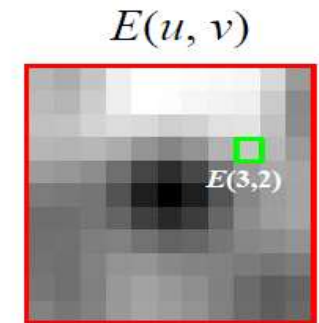
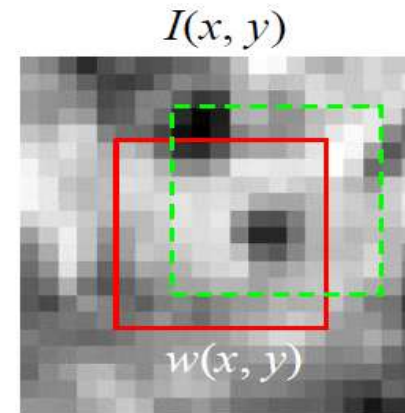
(u, v) 만큼의 이동에 대하여 밝기 분포가
특정 영역 윈도우 $w(x, y)$ 내에서 얼마나
변화하는 지를 계산함

$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

윈도우
함수

이동 후
밝기값

밝기값



- 특정 영역 : image 내부에서 윈도우 함수를 통해 구해진 영역입니다. 그림에서는 붉은 네모 내부의 image 영역입니다.
- $E(u, v)$: 특정 영역의 image를 x축 방향으로 +u , y축 방향으로 +v 만큼 이동하여 녹색 네모를 형성합니다. 그 후 수식을 통해 붉은 네모와 녹색 네모의 밝기 분포의 차이를 계산합니다. 차이를 계산하기 때문에 제곱을 해줍니다.
- 우측 그림은 (3,2)만큼 붉은 네모 image를 이동시키고 $E(3,2)$ 를 구하는 그림입니다.
- $E(u, v)$ 를 계산하면 두 영역의 밝기 분포 차이를 알 수 있고 , 밝기 분포 차이의 변화가 큰 영역이 특징점이 됩니다. 그러므로 $E(u, v)$ 가 크면 Corner Point일 확률이 높습니다.

<7주차 : 다양한 특징 검출 기법>

*** 인강 들어도 이해 불가 ***

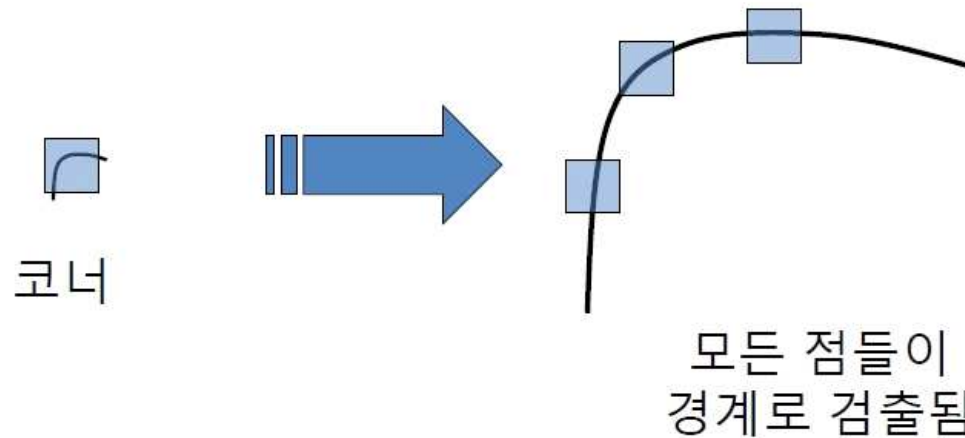
1. Invariance

= 영상의 밝기 변화에도 검출된 corner point의 위치가 변화하지 않습니다.

2. Covariance

= 영상 내 물체의 위치 변화에도 검출된 corner point의 위치가 변화하지 않습니다.

- 코너 검출은 Covariant 합니다.
- 코너 검출은 물체의 회전에 covariant 합니다.
- 코너 검출은 물체의 확대(Scaling)에 대해서는 covariant 하지 않습니다.
- 코너가 확대 되면 모서리가 생깁니다.



3. SIFT(Scale Invariant Feature Transform)

= Image의 크기(Scale)와 관련 없이 Image에서 특징점을 잘 찾아내는 기법입니다.

- Image를 단계적으로 축소시켜서 일련의 영상을 생성합니다. (점점 저해상도) 이 과정에서 다양한 크기의 Image가 만들어지고 각각의 corner를 검사합니다.

1) 스케일-공간 극점 검출

= 각각의 크기마다 영상마다 laplacian 값 계산

- 해바라기에서 scale이 커지면 더 잘보임

- laplacian은 느려서 DoG를 이용해서 계산 DoG는 laplacian이랑 비슷한데 속도가 빠름 그리고 거의 비슷함

2) 특징점 지역화 및 필터링

- 낮은 극값을 갖는 애들은 제거

- Edge 응답을 갖는 특징점 제거

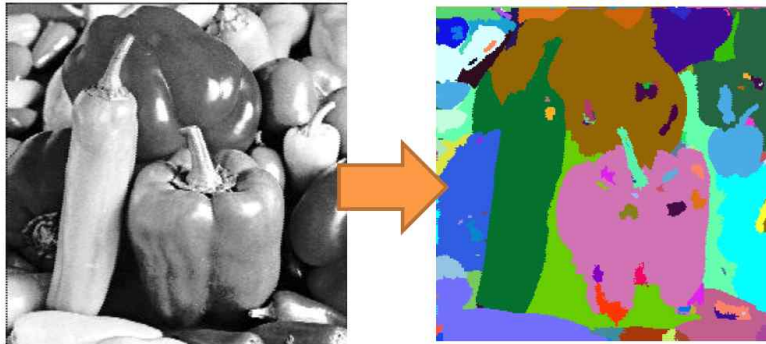
3) 방향 할당

- 그래디언트 크기 및 방향 계산

4) SIFT 기술자 생성

2. SURF(Speeded Up Robust Features)

<8주차 : 영상 분할>



1. 영상 분할의 정의

= 영상을 영역으로 구분합니다.

- 영상을 영역으로 구분하여 만든 데이터는 다양한 분야에서 사용됩니다.

2. 영상 분할이 어려운 이유 : 부분 체적 효과

부분 체적 효과

- 촬영 장비(CT scanner 등)의 제한된 해상도로 인하여 서로 다른 물질 간의 경계 영역에서 하나의 pixel/voxel에 여러 물질이 동시에 영향을 미치는 현상임
- 부분 체적 효과의 영향을 받는 pixel/voxel에서 촬영된 밝기값은 개별 물질들이 각 partial volume ratio로 혼합된 결과임



이상적인 영상

부분 체적 효과

- 부분 체적 효과로 인해 서로 다른 물체 간의 경계 영역에서의 구분이 뚜렷하지 않습니다.

3. 4가지 영상 분할의 기법

1) 밝기 값-기반 영상 분할 기법 : Thresholding

- 임계값에 따라서 영상의 물체를 분할합니다.

① Adaptive Thresholding

= 컴퓨터가 최적화된 임계값을 찾아서 image를 분할 하는 방식

② 씨앗점(특정한 점) 영역 성장법

= 씨앗점을 기준으로 인접하고 유사한 조건을 갖는 점들에 대해 재귀적으로 확장

- Leakage가 발생하기 쉽습니다.

2) 기울기-기반 영상 분할 기법 : Watershed Segmentation

= 기울기가 작은 부분을 모아서 분할

- 과분할(=과도하게 잘게 분할)이 발생합니다.

- 다양한 시그마 값에 따라서 영상이 나오는데 , 시그마를 크게 주면 과분할이 줄어듭니다.

3) 변형-모델 기반 영상 분할 기법 : 액티브 컨투어 모델

- 수학적으로 에너지 곡선 만들어서 사용.

- 수학식은 참고만

4) 통계-모델 기반 영상 분할 기법 : Atlas(사회과부도의 지도) 기반 분할

- 단일 Atlas가 존재하는 경우 , 해당 Atlas를 영상에 적용하여 분할합니다.

- 여러 가지 Atlas가 존재하는 경우 , 다음의 3가지 방식 중에서 선정하여 분할합니다.

① 가장 유사한 atlas를 선택하여 분할합니다.

② Atlas들의 평균을 낸 평균 Atlas를 이용해서 분할합니다.

③ 각각의 Atlas를 영상에 적용하여 분할을 하고 , 결과들을 Decision Fusion을 하여 최종 분할 결과를 나타냅니다.

4. 자동 분할 vs 반자동 분할

- 그래프 기반 영상 분할 기법은 자동 분할 OR 반자동 분할로 나눌 수 있습니다.

1) 자동분할

- 개발자의 입장에서는 간편하지만, 100% 자동 분할은 불가능합니다.
- 현대에는 인공지능의 등장으로 획기적으로 성능이 향상 중입니다.

2) 반자동 분할 기법

- 개발자의 입력이 필요합니다.
- 영상 분할 기법 중에서 가장 많이 사용됩니다.

5. 반자동 분할 기법 기반 + 그래프 기반 영상 분할 기법 : Graph-Cut 반자동 분할 기법

1) 정의 및 특징

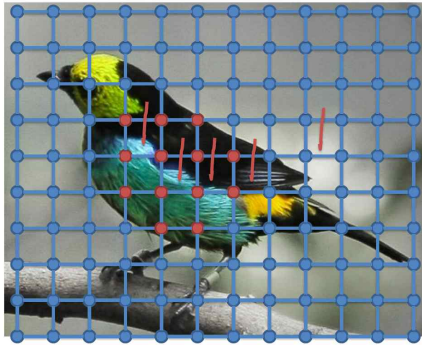
= 영상을 그래프로 나타냅니다.

- 영상을 물체(=객체)와 배경으로 구분합니다. 이때 물체에 해당하는 pixel은 0, 배경에 해당하는 pixel은 1을 할당합니다.
- pixel이 Node가 되고 pixel끼리의 연결선이 edge(가중치 있음)가 됩니다.
- 가중치의 예로는 'RGB의 밝기 값 차이'가 존재합니다.
- 다양한 씨앗점 입력에 대하여 안정적인 결과를 도출합니다.
- Cost Function을 제작합니다.
- Graph-Cut 방식을 사용하면 탐색 속도가 빠릅니다.
- 3D 영상을 그래프 기반 영상 분할한 영상을 보여주셨습니다.

2) 예시

① 적은 가중치

- 붉은 선으로 표시한 edge에서 두 pixel 간의 밝기 값 차이가 작습니다.



해당 cut은 적은 가중치

$$W(i,j) = |\text{RGB}(i) - \text{RGB}(j)|$$

② 높은 가중치

- 붉은 선으로 표시한 edge에서 두 pixel 간의 밝기 값 차이가 큼니다.

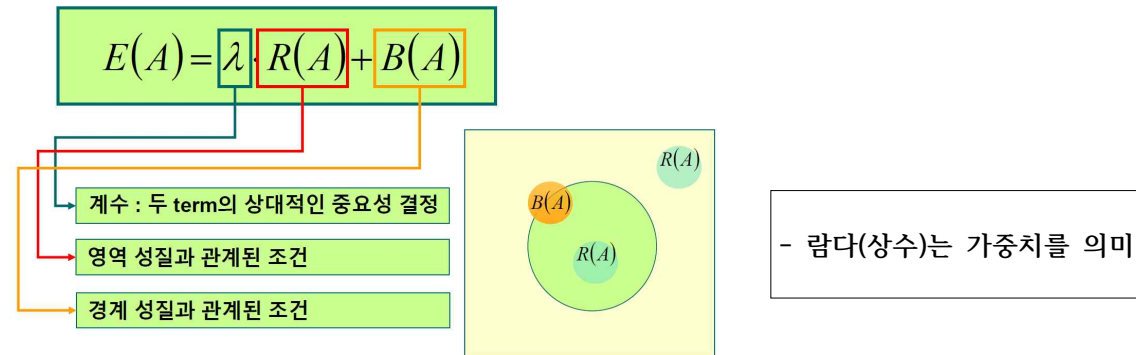


해당 cut은 높은 가중치

$$W(i,j) = |\text{RGB}(i) - \text{RGB}(j)|$$

3) cost Function

- Cost function



4) 영역 성질과 관계된 조건 : $R(A)$

= 어떤 점이 물체나 배경으로 할당되면 그때의 페널티를 숫자로 나타냄.

5) 경계 성질과 관계된 조건 : $B(A)$

- 하나는 물체 , 다른 하나는 배경
- 경계는 밝기 값 차이가 큼.
- 경계는 텍스처가 다릅니다.

<9주차 : OpenCV>

1. OpenCV

= Open Source Computer Vision Library

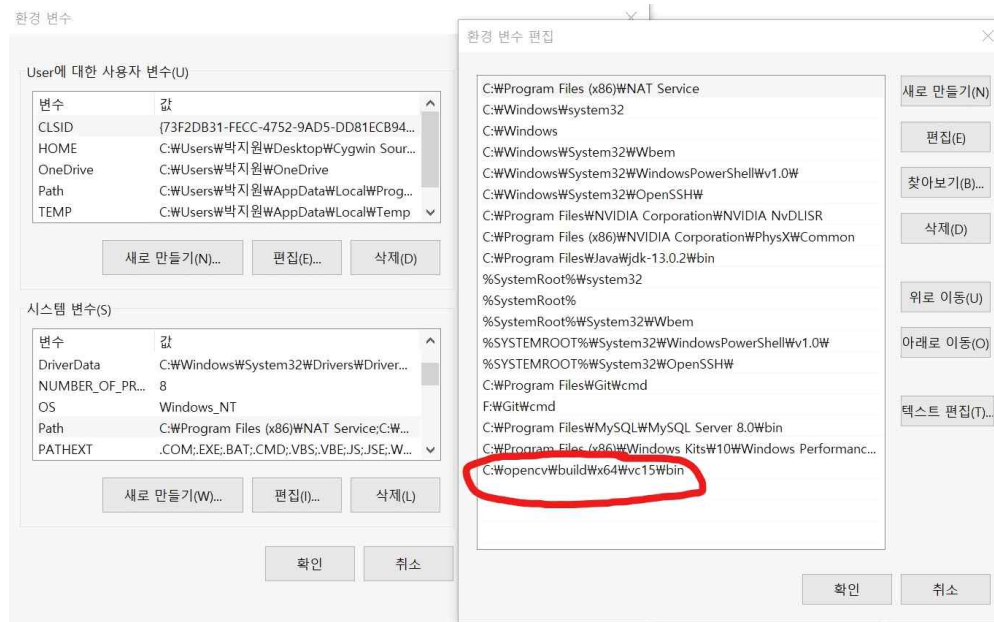
= 영상 처리와 컴퓨터 비전 관련 오픈소스 라이브러리

- C , C++ , Python , MatLab에서 사용

2. OpenCV 설치

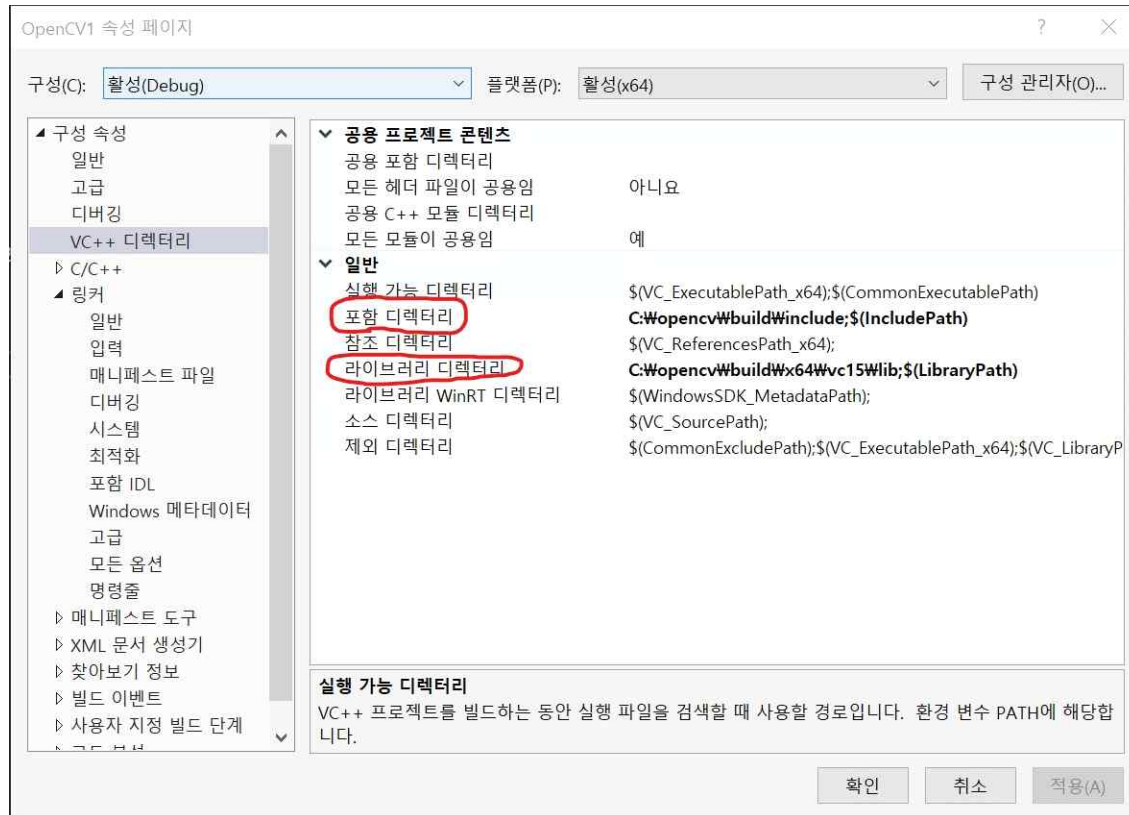
- DLL : Dynamic Linking Library : 프로그램 내에서 함수 호출 정보만 포함하고 목적 코드를 실행 시간에 호출

1) 환경 변수 -> 시스템 변수 -> Path에 경로 추가



- vc15는 버전별로 폴더가 다르기 때문에 x64 폴더에서 가장 번호가 높은 vc를 경로로 지정합니다.

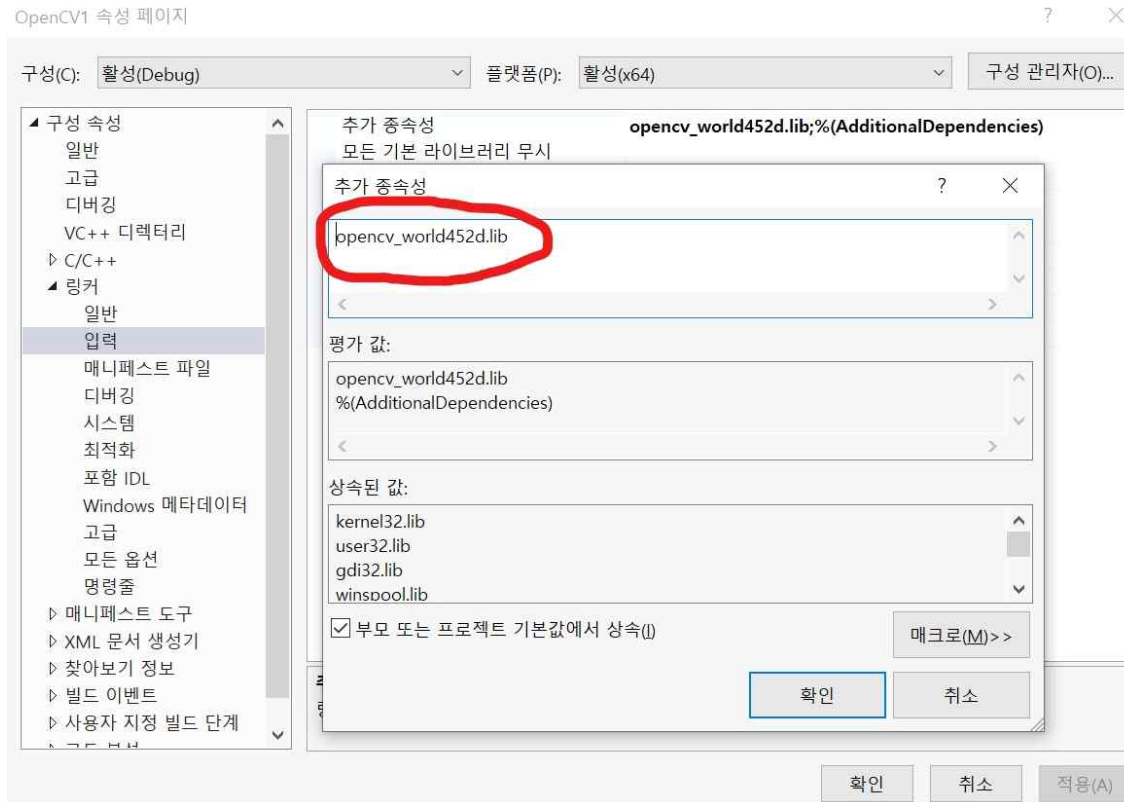
2) 프로젝트 속성 -> VC++ 디렉터리 -> 포함 디렉터리 & 참조 디렉터리



- 포함 디렉터리 : include 폴더

- 라이브러리 디렉터리 : x64 -> 번호 높은 vc -> lib

3) 프로젝트 속성 -> 링커 -> 입력 -> 추가 종속성



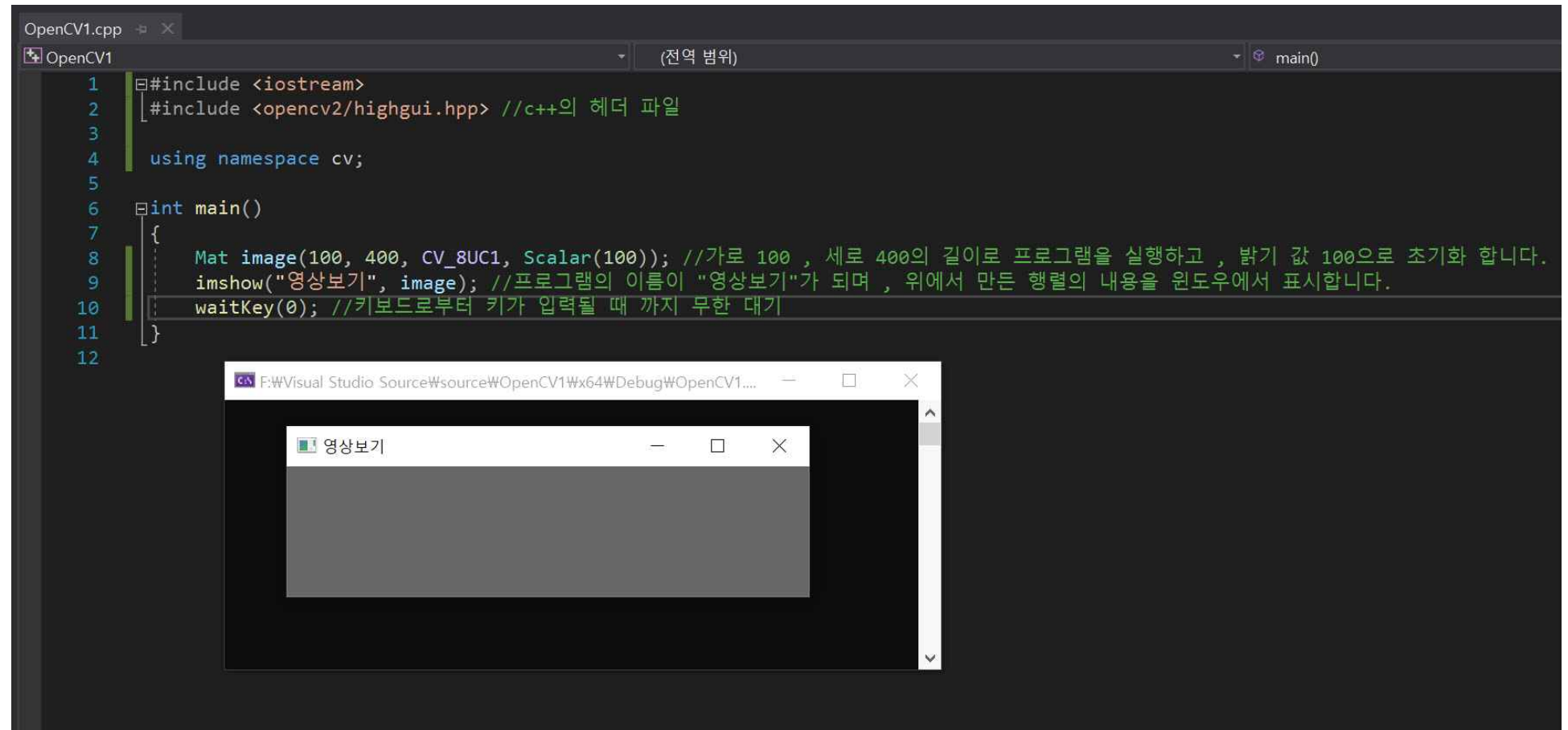
- x64 -> 번호 높은 vc -> lib에 들어가면 opencv_world 파일이 존재합니다.

- opencv 버전마다 해당 파일의 버전 번호가 다르므로 해당 하는 번호의 파일을 적고 , 반드시 d가 붙은 파일로 합니다.

- .lib파일로 적어주면 잘 실행되지만 .dll파일로 적어주면 경로에서 찾지 못하고 실행 ERROR가 발생합니다.

*** 2)번 과정과 3)번 과정은 Visual Studio 전체에 적용되는 설정이 아니므로 , 어떤 설정(교수님 강의에 있음)을 하지 않으면 매 프로젝트 마다 설정해 주어야 합니다. ***

3. 간단한 예제 프로그램



4. 헤더 파일과 라이브러리 파일

*** 중요하니까 컴구에도 옮겨서 적자 ***

1) Header file

Header Files : The files that tell the compiler how to call some functionality (without knowing how the functionality actually works) are called header files. They contain the function prototypes. They also contain Data types and constants used with the libraries. We use **#include** to use these header files in programs. These files end with **.h** extension.

Example : Math.h is a header file which includes the prototype for function calls like `sqrt()`, `pow()` etc, whereas **libm.lib**, **libmmd.lib**, **libmmd.dll** are some of the math libraries. In simple terms a header file is like a visiting card and libraries are like a real person, so we use visiting card(Header file) to reach to the actual person(Library).

- Example을 해석하면 매우 쉽게 Header file과 Library의 차이를 이해할 수 있습니다.
- = 컴파일러가 어떤 함수를 호출하면 되는지 적어 놓은 파일입니다.
- 함수의 원형을 적어놓습니다.
- 개발자가 .cpp와 .h를 구분해서 코딩을 진행할 때 자신이 사용할 함수를 .cpp에서 구현하고 .h에 원형만 저장합니다. 이와 동일하게 library에서 구현해놓은 함수는 windows의 라이브러리 파일(lib & dll 파일)에 저장되어 있고 , 헤더 파일에 그 함수들의 원형이 적혀있습니다.

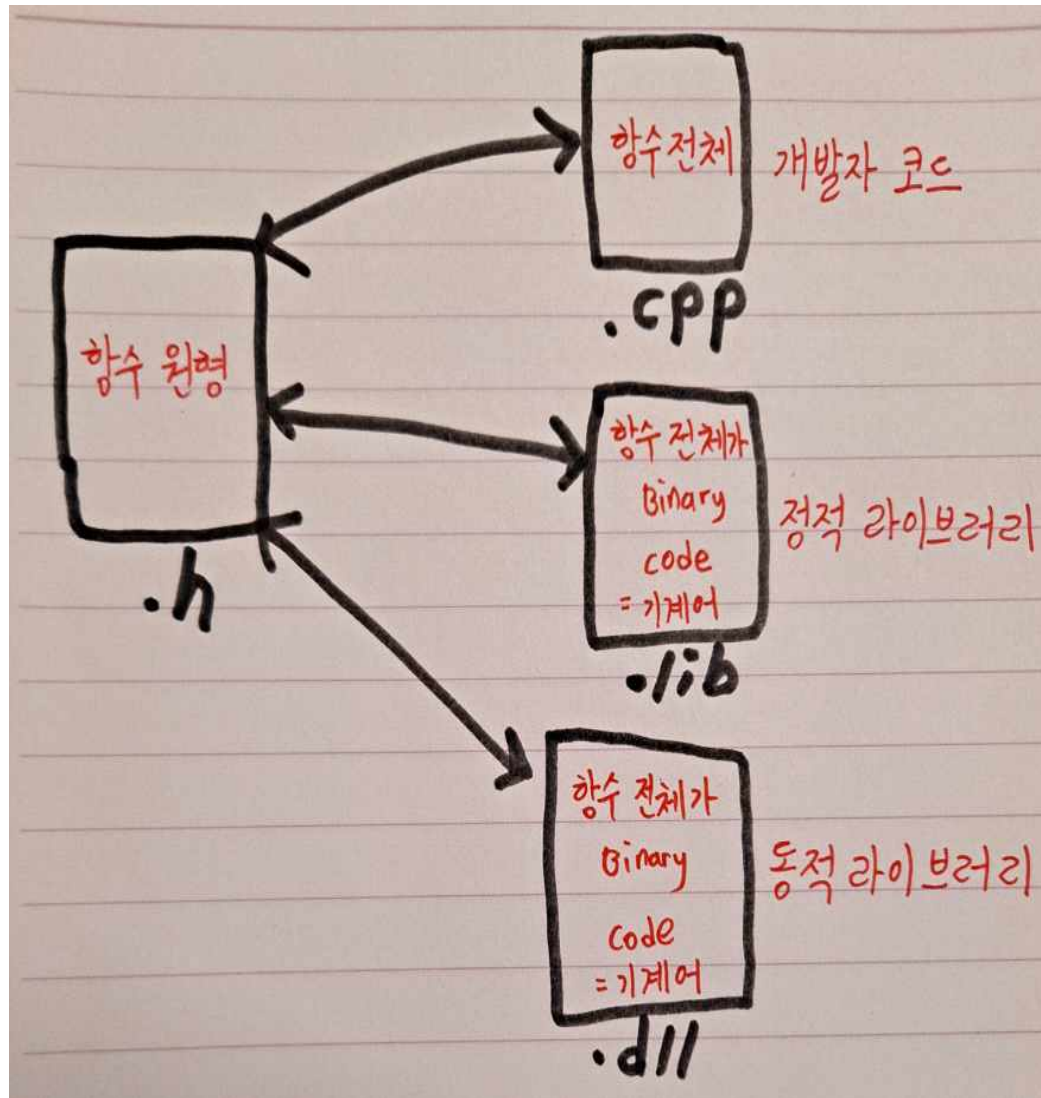
<내일 공부>

https://wnsgml972.github.io/setting/2018/11/01/dll_lib/

<https://goodgid.github.io/Static-VS-Dynamic-Library/>

<https://www.geeksforgeeks.org/difference-header-file-library/>

<https://coding-chobo.tistory.com/64>



<11주차 : 특징 기술자>

1. Color

1) 색의 기초

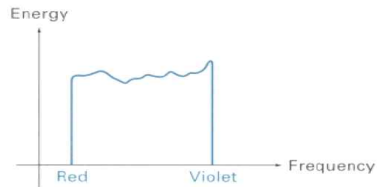
① 명도 : 색의 밝은 정도



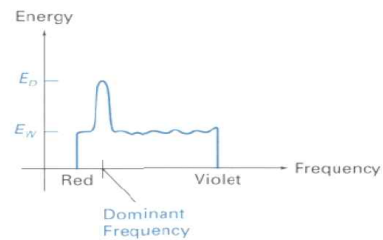
② 채도 : 색의 탁한 정도이며 , 원색(물감 12색)에 가까울수록 채도가 높습니다.



2) Visible Light를 나타내는 HSL Model



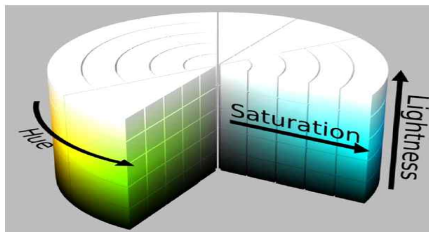
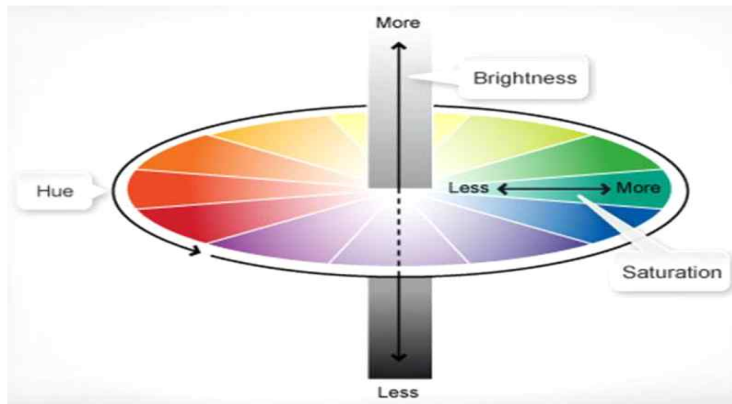
White Light



Orange Light

- 사람이 볼 수 있는 빛의 스펙트럼을 진동수로 나타내었습니다.
- **진동수(Frequency, HZ단위)와 파장(람다)은 반비례 관계**입니다.
- 작은 진동수이며 큰 파장을 갖는 색은 Red입니다.
- 큰 진동수이며 작은 파장을 갖는 색은 Violet입니다.

<HSL Model>



① H : Hue

= 색상

ex) Orange : 오렌지 부분의 energy가 확 올라갑니다.

② S : Saturation

= 채도

③ L : Lightness (=Brightness)

= 명도

= 그래프의 면적

3) Color Model

= 각각의 모델의 멤버 변수들의 값을 섞어서 색을 표현합니다.

① RGB : 기본 삼원색 (Red , Green , Blue)

② XYZ : 인간은 파장마다 다르게 받아들이는 원추세포가 존재합니다. 이를 기반으로 만들어진 Color Model입니다.

③ CMY : 고화질 인쇄에서 사용하는 삼원색 (Cyan , Magenta , Yellow)

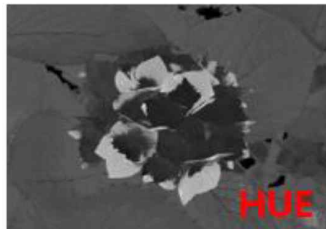
④ HSV : 실제 컬러 영상에서 처리할 때 유리한 색 모델

- H(Hue) : 색상 , 0에서 360사이의 값을 가집니다.

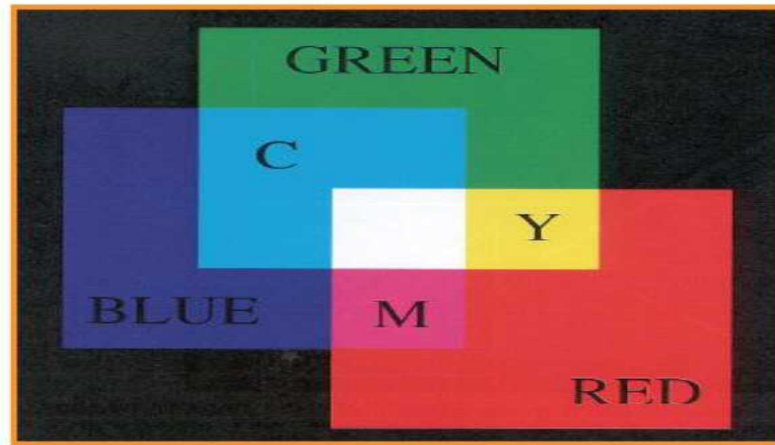
- S(Saturation) : 채도 , 0에서 100사이의 값을 가지며 0에 가까울수록 무채색에 가깝습니다.

- V(Value) : 명도 , 0에서 100사이의 값을 가지며 0은 검정이고 100은 흰색입니다.

<HSV의 요소 3가지를 각각 변경했을 때>



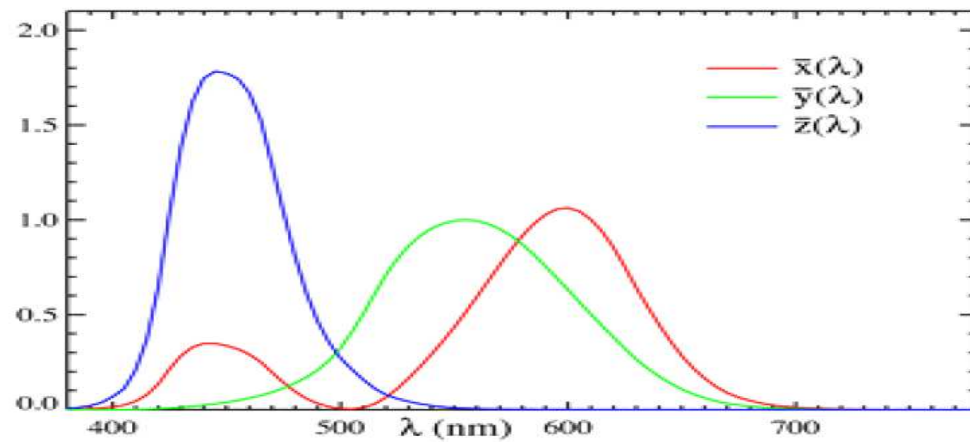
4) Color Model Example



RGB



CMY



XYZ

Red color (R)

7

Green color (G)

255

Blue color (B)

30

Convert Reset Swap

Hex color code

#07FF1E

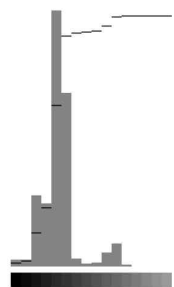
RGB color code

rgb(7,255,30)

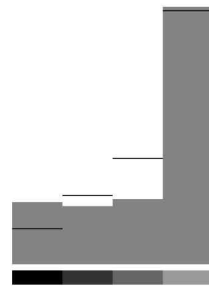
RGB를 섞어서 만든
상수 Hex 값

2. 컬러 기반 영역 특징 기술자

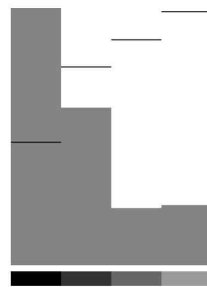
- RGB를 HSV로 변경해서 처리 합니다.
- HSV 컬러 히스토그램을 만들 수 있습니다.
- Binning(특징 벡터의 크기 감소), Clustering(잡음 감소)
 - Binning: Hue (16), Saturation (4), Value (4)



Hue Histogram
after Binning



Saturation Histogram
after Binning



Value Histogram
after Binning

- Hue의 값은 16단계로 구분했습니다.
- Saturation의 값은 4단계로 구분했습니다.
- Value의 값은 4단계로 구분했습니다.

<Color Coherence(통일) 벡터>

- color에 대해서 coherent인지 incoherent인지로 구분

coherent 픽셀 = 큰 연결 요소

incoherent 픽셀 = 작은 연결 요소

C : coherent 픽셀 개수

N : Incoherent 픽셀 개수

- 영상 크기의 1%로 tau 임계값을 설정하고 tau보다 크거나 같으면 coherent

13	15	11	17	15
27	10	12	25	22
23	4	25	11	9
29	3	21	23	8
1	9	24	2	7

30 colors

Color	0	1	2
C	8	8	6
N	0	0	3

1	1	1	1	1
2	1	1	2	2
2	0	2	1	0
2	0	2	3	0
0	0	2	0	0

3 colors

tau is 4

이거 이해 못함.

3. 형태 기반 영역 특징 기술자

- 대충 설명하시고 넘어 가셨습니다.

4. 중심 모멘트

- 모멘트 = 수학식

- 영상의 특징을 표현하기 매우 좋음.

이거 이해 못함

5. Zernike 모멘트

6. Texture (질감)

- = 통계학적 성질을 따르는 영상
- 비슷한 구조가 영상 내에서 계속 반복됩니다.

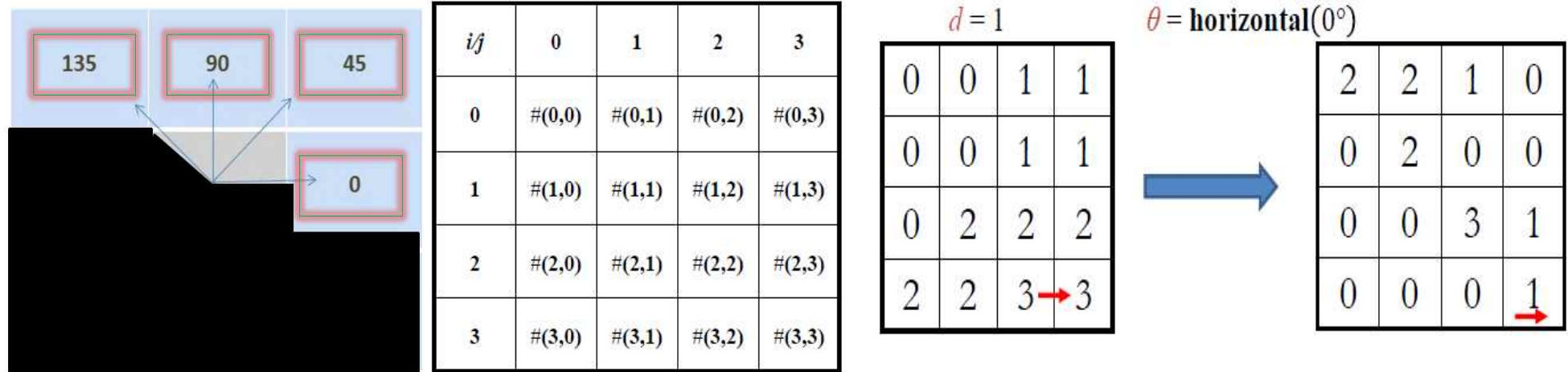


7. Texture 특징 기술자

1) GLCM (=Gray Level Co-occurrence Matrix)

- = 두 개의 픽셀이 일정한 거리를 두고 나타나는 빈도수를 각도마다 계산한 행렬
- 픽셀 쌍 사이의 상대적인 거리, 상대적인 각도가 필요합니다.
- 중심 픽셀을 기준으로 8방향이 모두 필요할 것 같지만 4방향 (0,45,90,135)만 필요합니다. 중심 픽셀 기준으로 위/아래를 변경하면 됩니다.
- GLCM을 통해 여러 가지 특징 지표를 제작합니다.
- 여러 가지 특징 지표는 '11주차_2차시 텍스처 특징 기술자 소개' PPT 15 ~ PPT 19를 참고 합니다.

<Example : GLCM 계산>



- d : 기준 pixel과 다른 pixel 간의 거리
- 행렬 1칸의 거리는 1입니다.
- θ : 기준 pixel과 다른 pixel 간의 각도
- 0도(=3시 방향), 45도(=1시 방향), 90도(=12시 방향), 135도(=11시 방향)

① 우측 행렬의 (0,0)이 2인 이유

= 좌측 행렬의 (0,0) ~ (3,3)까지 16개의 모든 pixel을 각각 기준으로 잡고 총 16번의 계산을 합니다. 각각의 계산은 기준 pixel에서 0도에 거리가 1인 pixel과 비교를 해서 값이 0에서 0으로 변화하였는지 검증합니다. 그 결과 좌측 행렬의 (0,0) -> (0,1) 그리고 (1,0) -> (1,1)의 값이 0에서 0으로 변화하였기 때문에 우측 행렬에서 (0,0)의 값이 2입니다.

② 우측 행렬의 (3,3)이 1인 이유

= 좌측 행렬의 (0,0) ~ (3,3)까지 16개의 모든 pixel을 각각 기준으로 잡고 총 16번의 계산을 합니다. 각각의 계산은 기준 pixel에서 0도에 거리가 1인 pixel과 비교를 해서 값이 3에서 3으로 변화하였는지 검증합니다. 그 결과 좌측 행렬의 (3,2) -> (3,3)의 값이 3에서 3으로 변화하였기 때문에 우측 행렬에서 (3,3)의 값이 1입니다.

2) LBP (=Local Binary Pattern)

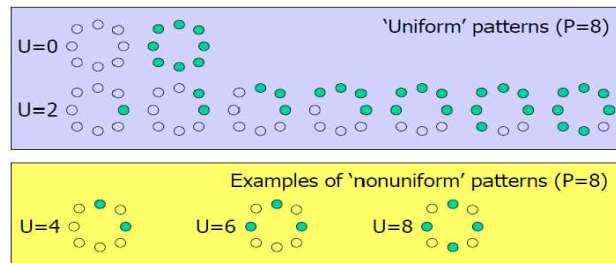
example	thresholded	weights																											
<table border="1"> <tr><td>6</td><td>5</td><td>2</td></tr> <tr><td>7</td><td>6</td><td>1</td></tr> <tr><td>9</td><td>8</td><td>7</td></tr> </table>	6	5	2	7	6	1	9	8	7	<table border="1"> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td></td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	1	0	0	1		0	1	1	1	<table border="1"> <tr><td>1</td><td>2</td><td>4</td></tr> <tr><td>128</td><td></td><td>8</td></tr> <tr><td>64</td><td>32</td><td>16</td></tr> </table>	1	2	4	128		8	64	32	16
6	5	2																											
7	6	1																											
9	8	7																											
1	0	0																											
1		0																											
1	1	1																											
1	2	4																											
128		8																											
64	32	16																											

Pattern = **11110001**

LBP = $1 + 16 + 32 + 64 + 128 =$ **241**

- = 주변 픽셀이 중심 픽셀에 비하여 밝기 값이 같거나 크면 1, 밝기 값이 작으면 0의 값을 갖고, 그것을 이진수로 표현하는 기법
- Noise와 illumination gradient에 민감합니다.
- Thresholded : 중심 pixel(1,1)에 대해서 총 8개의 이진수 값이 생성됩니다.
- Weights : 이진수의 자릿수를 표현합니다.
- 이진수의 Weights(자릿수)를 참고해서 Pattern을 작성하면 11110001(2)가 나오며 이를 10진수로 변환하면 LBP 값인 241(10)이 결과로 도출됩니다.

<Uniform pattern>

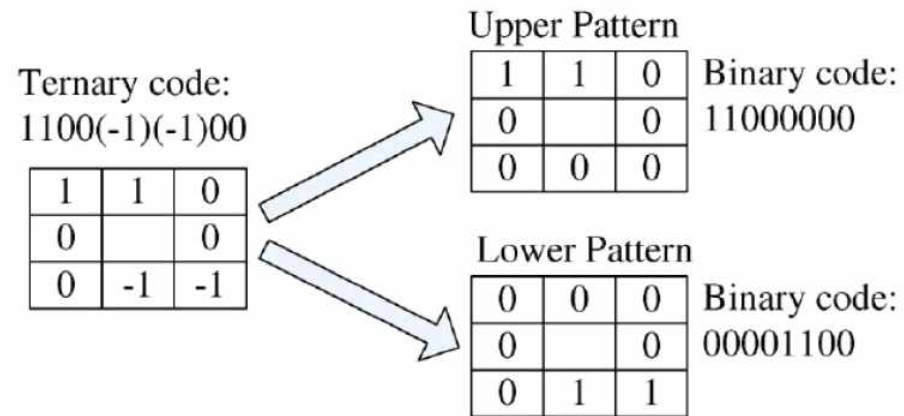


- 원형으로 구성된 LBP
- 인접하면 'Uniform' Patterns 입니다.
- 인접하지 않으면 'Non-Uniform' Patterns 입니다.

3) LTP (=Local Ternary Pattern)

= LBP의 업그레이드 버전

- 중심 pixel을 기준으로 범위를 두고 값을 -1, 0, 1로 매깁니다.

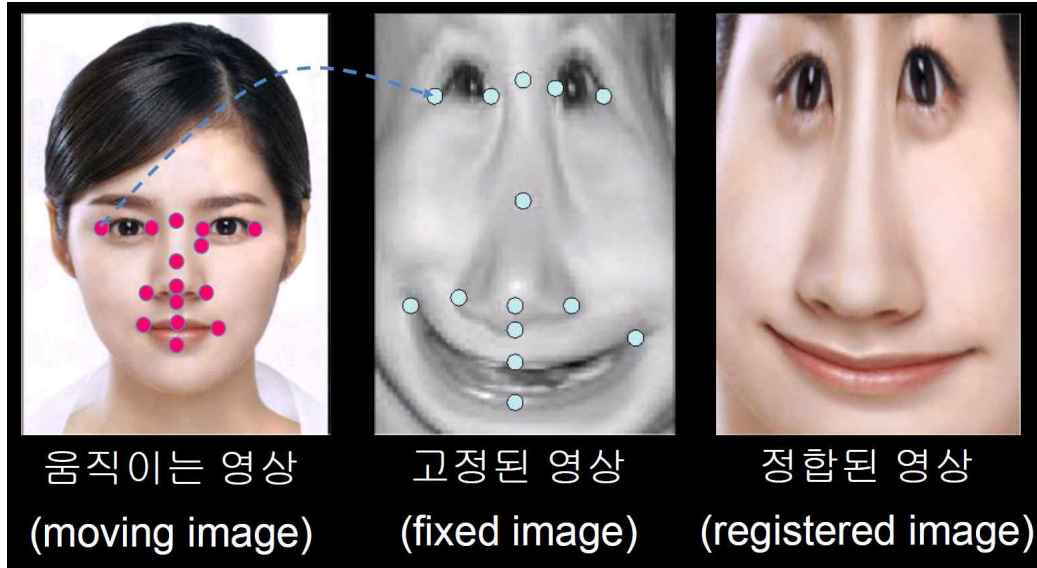


① upper pattern : 값이 1인 픽셀만 1로 , 나머지는 0으로 표기

② lower pattern : 값이 -1인 픽셀만 1로 , 나머지는 0으로 표기

<12주차 : 영상 정합 기술>

1. 영상 정합 (=영상 매칭)



- 아이의 얼굴 모양에 여성의 얼굴을 mapping 시켜 영상 정합을 실행 시킵니다.

= 하나의 장면이나 물체를 다른 시간이나 관점에서 촬영할 경우, 영상은 서로 다른 좌표계에서 얻어지게 됩니다. 영상 정합은 이와 같은 서로 다른 영상을 변형하여 하나의 좌표계에 나타내는 처리기법입니다.

= 서로 다른 두 영상 사이의 같은 부분(=대응 관계)을 찾아내는 기법

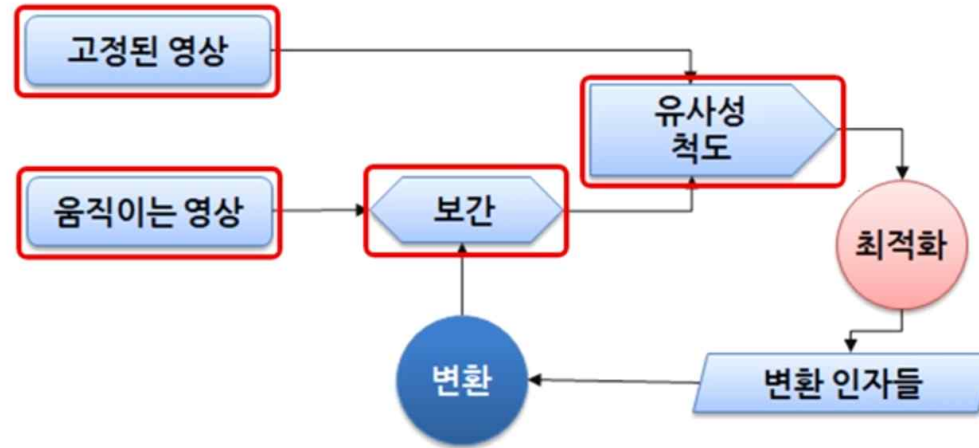
- 정합을 통해 서로 다른 측정 방식을 통해 얻은 영상이 어떻게 대응되는지를 알 수 있다.

ex) 눈 끝점 코 입술을 어린아이의 얼굴처럼 이동시킴

ex) 환자의 환부 영상을 시간에 따라 비교

ex) 환자의 환부 영상과 정상인의 영상을 비교

<영상 정합 과정>



2. 변환 모델

1) 이동, 확장, 회전

= 기존에 배운 기하 변환입니다.

- 원점을 기준 변환합니다.

2) 어파인 변환

= 위의 개념에 전단(shearing) 까지 포함하는 개념

- 전단이 뭔지 식을 보여줌. ppt

- 전단은 x축 방향으로 밀고, y축 방향으로 미는 걸 포함합니다.

3) 편미분 방정식 기반 모델

- 고급 주제라 간단하게만 소개하고 넘어가셨습니다.

ex) 박판 스플라인, 유체 역학 기반, b-스플라인, 유한 요소, 방사상 기저 함수

3. 밝기 값 기반 영상 정합

= 영상의 밝기 값을 사용합니다.

- 두 영상의 중첩된 부분(= Ω = 오메가)만 밝기 값을 비교합니다.
- 두 영상의 중첩된 부분에서만 , 두 영상의 밝기 값 차이의 제곱으로 오차를 구합니다.
- 중첩된 영역이 없을 경우 오차가 0이 나옵니다. 그렇기 때문에 영상 정합 초기에 두 영상이 중첩되는 영역이 존재하도록 설정.
- 최적화 과정 : 영상 정합을 계속 반복하다 보면 점점 오차가 줄어들고 영상 정합의 성능이 높아집니다.

<오차를 사용하는 방식의 장단점>

장점	단점
▶ 계산하기 간단함 ▶ 미분이 가능함	▶ 영상 간 카메라 차이나 조명 차이에 의한 오차를 보정하기 어려움 ▶ 의료 영상에서는 조영제 등에 의한 오차를 보정하기 어려움

4. 특징 기반 영상 정합

- 특징 점을 이용하는 기술
- 특징 점들 사이의 거리 , 두께 , 각도 차이를 측정합니다.

5. 표면 정합 기법

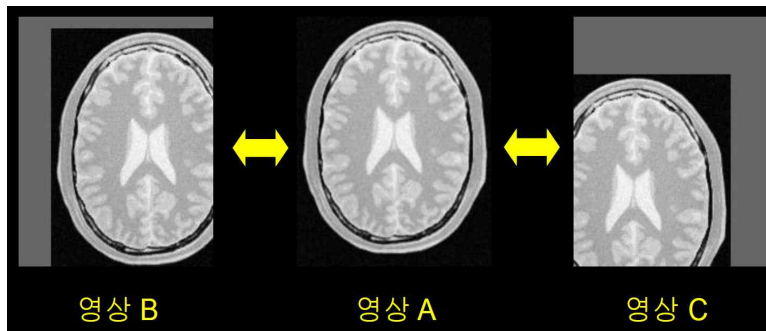
- 어려운 주제이므로 간단히 설명하고 넘어가겠습니다.

1) Point Signature

2) PCA (Principal Component Analysis)

3) ICP (Iterative Closest Point)

6. 유사성 척도



- 영상 A는 영상 B와 유사할까 , 아니면 영상 C와 유사할까
- 이 질문에 해답을 컴퓨터가 내리기 위해 유사성을 판단하는 척도가 필요합니다.

= 컴퓨터가 두 영상의 유사성을 판단하는 척도

- 움직이는 영상의 경우 계속 이동시켜서 유사성이 가장 높은 상태를 찾아야 합니다.

1) 평균 제곱 차이 : 오차

= 두 영상이 일치하면 값이 0이 됩니다.

2) Normalized Correlation

3) Mutual Information

= 대응되는 밝기 값의 쌍의 개수를 세는 방식

- 대응 되는 밝기 값의 쌍의 개수가 작으면 유사성이 높습니다.

- 두 영상의 촬영 장비(ex : 현미경 , CT , MRI)가 동일하지 않을 때 사용합니다.

7. 최적화 과정 (=오차 최소화)

= 미분해서 값이 0이 되는 지점을 찾는 과정입니다.

<15주차 특강 : 바이오메디컬 영상 분야를 위한 인공지능 기법 활용>

1. Deep Learning

- = 은닉 층이 여러 개 있는 층이 깊은 신경망
- 2017년에 사람의 인식을 보다 높은 성능
- Python과 API 사용법만 익히면 매우 쉽게 구현할 수 있습니다.
- 양질의 훈련 데이터 확보가 어렵습니다.
- 고성능의 GPU가 필요합니다.
- 내부 구조 및 weight 들의 의미 파악이 어렵기 때문에 ERROR 수정이 어렵습니다.

2. Deep Learning(큰 개념으로는 기계학습)이 왜 필요한가?

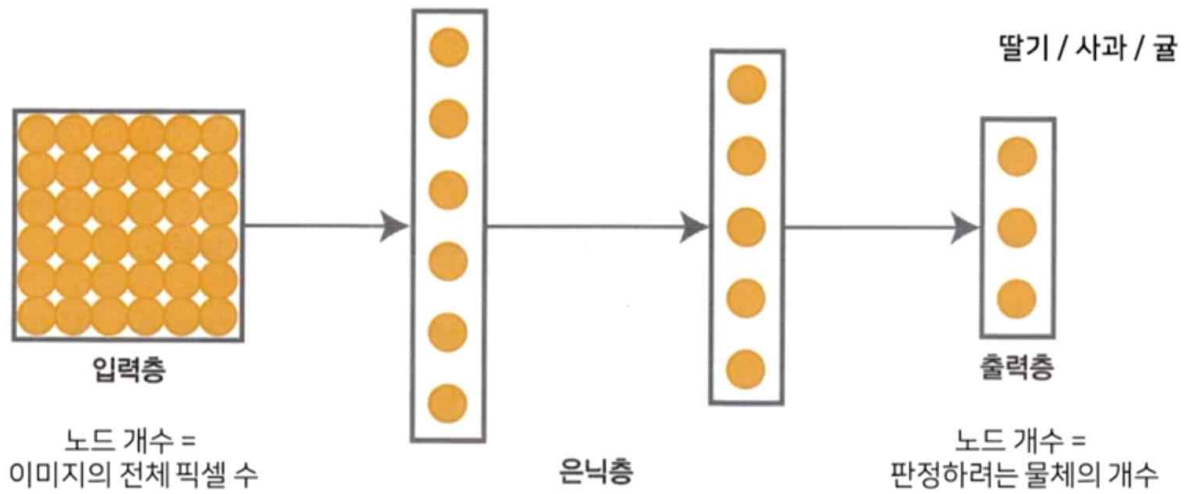
- 컴퓨터는 원래 순차적으로 처리함
- 사과를 찾을 때 , 빨간색의 동그란 물체가 있으면 사과 , 근데 이러면 토마토도 찾음
- 사과가 있는 영상과 , 사과가 없는 영상을 대량으로 준비하고 학습시킵니다.
- 데이터가 판단함

3. 인공 신경망

뇌 신경망	인공 신경망
뉴런	노드
수상 돌기, 축삭 돌기	웨이트
시냅스	활성화 함수

- 뇌의 신경망을 모델로 만듭니다.
- 노드와 노드를 연결.

- 입력층 -> 은닉층 -> 출력층
- <Example : 딸기/사과/귤 판별>



- 입력 층의 노드의 개수 = 이미지의 전체 픽셀 개수
- 출력 층의 노드의 개수 = 판정하려는 물체의 개수